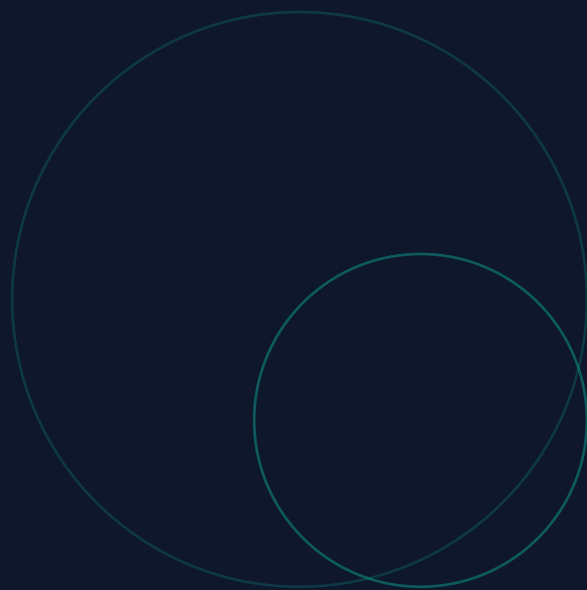


ENGENHARIA AGÊNTICA APLICADA

O Grande Livro Visual da Auditoria AIDD

A Fábrica de Software Perfeita: Uma Viagem Pelas 8 Ferramentas Macro, 44 Micro-Ferramentas, 65 Guardas e 132 Componentes

Do macro ao micro: o ecossistema como um todo, os três fluxos canônicos de criação e cada uma das oito ferramentas — arquitetura, engenharia agêntica e economia de tokens, camada por camada.



Equipe de Engenharia Canônica do Ecossistema AIDD e Antigravity Agent

4 de outubro de 2026

SOBRE ESTA OBRA

Este livro é a documentação visual e pedagógica definitiva de cada engrenagem do Ecossistema AIDD. Escrito através da técnica dual (Na Festa para explicar de forma lúdica que até uma criança entenda, e Na Casa para fornecer o rigor técnico, os caminhos dos arquivos no disco e as 15 dimensões da Lente 15-D, preenchidas com evidência lida do repositório). Cobre com precisão cirúrgica as 8 Ferramentas Macro, as 44 Micro-Ferramentas (Skills), os 65 Quality Gates determinísticos e os 132 Componentes Agnósticos sincronizados em múltiplos ambientes.

Sumário

Prólogo: Bem-vindo à Cidade da Fábrica de Brinquedos Perfeita	6
PARTE I: AS 8 FERRAMENTAS MACRO (AS GRANDES OFICINAS)	7
Como ler as fichas desta parte: 11 dimensões antigas, 15 dimensões atuais	7
Capítulo 1: AIDD Forge — O Altar da Fundação e os Guardiões das Leis	7
Na Festa (A Metáfora Memorável)	7
Na Casa (A Foto Técnica e a Ficha Histórica de 22/09/2026)	7
Ficha de Auditoria Bit a Bit: aidd-forge	9
SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)	9
1. Matriz Contratual das 11 Dimensões	9
2. Evidência de Testes e Portões	10
3. Diagnóstico de Não-Conformidades	10
SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA	10
SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)	10
Capítulo 2: AIDD Planner — A Mesa do Arquiteto e o Livro de Receitas	10
Na Festa (A Metáfora Memorável)	10
Na Casa (A Foto Técnica e a Ficha Histórica de 22/09/2026)	10
Ficha de Auditoria Bit a Bit: aidd-planner	11
SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)	11
1. Matriz Contratual das 11 Dimensões	11
2. Evidência de Testes e Portões	12
3. Diagnóstico de Não-Conformidades	12
SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA	12
SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)	12
Capítulo 3: AIDD Generator — O Trem Autônomo de 8 Vagões (Fluxo 01: Do Zero Puro)	12
Na Festa (A Metáfora Memorável)	12
Na Casa (A Foto Técnica e a Ficha Histórica de 22/09/2026)	12
Ficha de Auditoria Bit a Bit: aidd-generator (Fluxo 01 — Do Zero Puro)	13
SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)	13
1. Matriz Contratual das 11 Dimensões	13
2. Evidência de Testes e Portões	14
3. Diagnóstico de Não-Conformidades	14
SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA	14
SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)	14
Capítulo 4: AIDD Factory — A Linha de Montagem de Peças Prontas (Fluxo 02: Open-Source)	14
Na Festa (A Metáfora Memorável)	14
Na Casa (A Foto Técnica e a Ficha Histórica de 22/09/2026)	14
Ficha de Auditoria Bit a Bit: aidd-factory (Fluxo 02 — Motores Open-Source)	16
SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)	16
1. Matriz Contratual das 11 Dimensões	16
2. Evidência de Testes e Portões	17
3. Diagnóstico de Não-Conformidades	17
SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA	17
SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)	17
Capítulo 5: AIDD Bridge — A Ponte da Libertação (Fluxo 03: Desacoplamento Low-Code)	17
Na Festa (A Metáfora Memorável)	17
Na Casa (A Foto Técnica e a Ficha Histórica de 22/09/2026)	18
Ficha de Auditoria Bit a Bit: aidd-bridge (Fluxo 03 — Low-Code / Desacoplamento)	19

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)	19
1. Matriz Contratual das 11 Dimensões	19
2. Evidência de Testes e Portões	20
3. Diagnóstico de Não-Conformidades	20
SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA	20
SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)	20
Capítulo 6: AIDD Master — O Maestro da Harmonização Monolítica VSA	20
Na Festa (A Metáfora Memorável)	20
Na Casa (A Foto Técnica e a Ficha Histórica de 22/09/2026)	20
Ficha de Auditoria Bit a Bit: aidd-master (Meso-Camada de Convergência Monolítica VSA)	22
SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)	22
1. Matriz Contratual das 11 Dimensões	22
2. Evidência de Testes e Portões	23
3. Diagnóstico de Não-Conformidades	23
SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA	23
SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)	23
Capítulo 7: AIDD Enterprise — O Cofre de Alta Segurança e Selos Criptográficos	23
Na Festa (A Metáfora Memorável)	23
Na Casa (A Foto Técnica e a Ficha Histórica de 22/09/2026)	23
Ficha de Auditoria Bit a Bit: aidd-enterprise (Resiliência Crítica SHA-256)	25
SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)	25
1. Matriz Contratual das 11 Dimensões	25
2. Evidência de Testes e Portões	26
3. Diagnóstico de Não-Conformidades	26
SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA	26
SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)	26
Capítulo 8: AIDD Ops — A Usina de Força e a Torre de Vigilância	26
Na Festa (A Metáfora Memorável)	26
Na Casa (A Foto Técnica e a Ficha Histórica de 22/09/2026)	26
Ficha de Auditoria Bit a Bit: aidd-ops (Infraestrutura, VPS e Observabilidade)	27
SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)	27
1. Matriz Contratual das 11 Dimensões	27
2. Evidência de Testes e Portões	28
3. Diagnóstico de Não-Conformidades	28
SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA	28
SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)	28
PARTE II: O CINTO DE UTILIDADES (AS 44 MICRO-FERRAMENTAS / SKILLS)	29
Na Festa	29
Na Casa (O Catálogo Completo das 44 Skills Canônicas)	29
1. Micro-Ferramenta: aidd-9router	29
2. Micro-Ferramenta: aidd-agent-writing	30
3. Micro-Ferramenta: aidd-anatomy	31
4. Micro-Ferramenta: aidd-audit-4f	31
5. Micro-Ferramenta: aidd-components	32
6. Micro-Ferramenta: aidd-dataviz	33
7. Micro-Ferramenta: aidd-delivery	33
8. Micro-Ferramenta: aidd-dependencies	34
9. Micro-Ferramenta: aidd-diagnose	35
10. Micro-Ferramenta: aidd-dispatch	36
11. Micro-Ferramenta: aidd-docx	36

12. Micro-Ferramenta: aidd-enterprise	37
13. Micro-Ferramenta: aidd-evolution	38
14. Micro-Ferramenta: aidd-forge	38
15. Micro-Ferramenta: aidd-freedom	39
16. Micro-Ferramenta: aidd-frontend-forms	40
17. Micro-Ferramenta: aidd-grill	40
18. Micro-Ferramenta: aidd-grill-docs	41
19. Micro-Ferramenta: aidd-handoff	42
20. Micro-Ferramenta: aidd-improvement	42
21. Micro-Ferramenta: aidd-ingest	43
22. Micro-Ferramenta: aidd-master	44
23. Micro-Ferramenta: aidd-mcp	45
24. Micro-Ferramenta: aidd-open	45
25. Micro-Ferramenta: aidd-ops	46
26. Micro-Ferramenta: aidd-orca	47
27. Micro-Ferramenta: aidd-orchestrate	47
28. Micro-Ferramenta: aidd-pdf	48
29. Micro-Ferramenta: aidd-pipeline	49
30. Micro-Ferramenta: aidd-plan	49
31. Micro-Ferramenta: aidd-planner	50
32. Micro-Ferramenta: aidd-pptx	51
33. Micro-Ferramenta: aidd-pure	51
34. Micro-Ferramenta: aidd-reexplain	52
35. Micro-Ferramenta: aidd-retro	53
36. Micro-Ferramenta: aidd-session	54
37. Micro-Ferramenta: aidd-skills	54
38. Micro-Ferramenta: aidd-spec	55
39. Micro-Ferramenta: aidd-tdd	56
40. Micro-Ferramenta: aidd-textbook	56
41. Micro-Ferramenta: aidd-tickets	57
42. Micro-Ferramenta: aidd-visual-maps	57
43. Micro-Ferramenta: aidd-wizard	58
44. Micro-Ferramenta: aidd-xlsx	59
PARTE III: OS 65 GUARDIÕES INCORRÚPTÍVEIS (QUALITY GATES)	60
Na Festa	60
Na Casa (A Matriz dos 65 Quality Gates Canônicos)	60
1. Guarda Incorruptível: G_aidd_diagnose.py	60
2. Guarda Incorruptível: G_aidd_enterprise.py	61
3. Guarda Incorruptível: G_aidd_forge.py	61
4. Guarda Incorruptível: G_aidd_grill.py	62
5. Guarda Incorruptível: G_aidd_spec.py	62
6. Guarda Incorruptível: G_aidd_tdd.py	63
7. Guarda Incorruptível: G_aidd_tickets.py	63
8. Guarda Incorruptível: G_amelhoria.py	64
9. Guarda Incorruptível: G_ANT_LOCKIN_LEGADO.py	64
10. Guarda Incorruptível: G_ARQUITETURA_DELIVERABLE.py	65
11. Guarda Incorruptível: G_CLI_HELP_CONSISTENCIA.py	65
12. Guarda Incorruptível: G_COMPONENTE_AGNOSTICO.py	66
13. Guarda Incorruptível: G_CONTRACT_ROT.py	66

14. Guarda Incorruptível: G_DEPENDENCIAS_PIN_HASH.py	67
15. Guarda Incorruptível: G_DETERMINISMO_LEI_1.py	67
16. Guarda Incorruptível: G_DISCIPLINA_TESTE_FERRAMENTA.py	68
17. Guarda Incorruptível: G_DISPATCH_PIPELINE_VSA.py	69
18. Guarda Incorruptível: G_DOCS_ROT.py	69
19. Guarda Incorruptível: G_DRIFT_ANALYZER.py	70
20. Guarda Incorruptível: G_DRIFT_NUCLEO_COMPARTILHADO.py	70
21. Guarda Incorruptível: G_ECOSSISTEMA_INTEGRIDADE.py	71
22. Guarda Incorruptível: G_ENV_ROT.py	71
23. Guarda Incorruptível: G_ESCRITOR_ATOMICO.py	72
24. Guarda Incorruptível: G ESTRUTURA_ESTADO.py	72
25. Guarda Incorruptível: G_FRONTEIRA_FERRAMENTAS.py	73
26. Guarda Incorruptível: G_FRONTEND_LAYERS.py	73
27. Guarda Incorruptível: G_GESTOR_SESSOES.py	74
28. Guarda Incorruptível: G_GRAPH_FIRST.py	74
29. Guarda Incorruptível: G_HADOLINT.py	75
30. Guarda Incorruptível: G_HANDOFF_MELHORIA.py	75
31. Guarda Incorruptível: G_HARNESS_COMPAT.py	76
32. Guarda Incorruptível: G_HONESTIDADE_ROTULO.py	77
33. Guarda Incorruptível: G_IDIOMA_LEI_4.py	77
34. Guarda Incorruptível: G_INFRA_COMPOSE.py	78
35. Guarda Incorruptível: G_ISOLATION_AUDIT.py	78
36. Guarda Incorruptível: G_LAYOUT_ENTREGA.py	79
37. Guarda Incorruptível: G_LEI_DECLARA_PORTAO.py	79
38. Guarda Incorruptível: G_LIVRO_EVIDENCIA.py	80
39. Guarda Incorruptível: G_LLM_PROMPT_SHIELD.py	80
40. Guarda Incorruptível: G_mapa_pecas.py	81
41. Guarda Incorruptível: G_MIGRATION_ROT.py	81
42. Guarda Incorruptível: G_NOVE_CAMADAS_MERCADO.py	82
43. Guarda Incorruptível: G_ORQUESTRADOR_SINCRONO.py	82
44. Guarda Incorruptível: G_PACOTE_CORE.py	83
45. Guarda Incorruptível: G_PIPELINE_HANDOFF.py	84
46. Guarda Incorruptível: G_PORTAO_PROVA_QUE_MORDE.py	84
47. Guarda Incorruptível: G_PROTOCOL_FALLBACK.py	85
48. Guarda Incorruptível: G_PROTOTYPE_REWRITE.py	85
49. Guarda Incorruptível: G_PROVA_SKILLS_POCKOCK.py	86
50. Guarda Incorruptível: G_QUARTETO_SINE_QUA_NON.py	86
51. Guarda Incorruptível: G_RESUMO_USUARIO.py	87
52. Guarda Incorruptível: G_SAIDA_BINARIA.py	87
53. Guarda Incorruptível: G_SEGREDOS.py	88
54. Guarda Incorruptível: G_SKILL_FORMATO.py	88
55. Guarda Incorruptível: G_SKILL_ROT.py	89
56. Guarda Incorruptível: G_STACK_PADRAO_OURO.py	89
57. Guarda Incorruptível: G_SUPPLY_CHAIN.py	90
58. Guarda Incorruptível: G_SYNC_CMD_ROT.py	90
59. Guarda Incorruptível: G_TEMPLATE_FORGE_ROT.py	91
60. Guarda Incorruptível: G_TEMPLATE_TANSTACK_OFFLINE.py	92
61. Guarda Incorruptível: G_TESTES_REAIS.py	92

62. Guarda Incorrúptível: G_TRANSACTION_LOG_LRU.py	93
63. Guarda Incorrúptível: G_UNIVERSAL_HARNESS.py	93
64. Guarda Incorrúptível: G_USER_FACING_PTBR.py	94
65. Guarda Incorrúptível: G_ZERO_HEADLESS.py	94
PARTE IV: OS 132 COMPONENTES AGNÓSTICOS (PEÇAS UNIVERSAIS LEGO)	96
Na Festa	96
Na Casa (A Sincronização dos 132 Componentes)	96
As 6 Famílias dos 132 Componentes:	96
PARTE V: AS AUDITORIAS EM ANDAMENTO (OS CICLOS 4F)	97
Na Festa	97
Na Casa (o estado real em 4 de outubro de 2026)	97
Errata das Fichas de 2026-09-22	103
Epílogo: A Orquestra Perfeita	104

Prólogo: Bem-vindo à Cidade da Fábrica de Brinquedos Perfeita

Imagine uma cidade onde existe uma fábrica de brinquedos mágicos. Mas não é uma fábrica qualquer onde as coisas quebram e ninguém sabe o motivo. É uma fábrica que constrói castelos digitais inteiros em poucos minutos!

Para que tudo funcione sem nenhum fio solto, a fábrica possui: 1. **8 Grandes Oficinas Mestras (As Ferramentas Macro)**: Cada uma com um mestre construtor responsável por uma grande missão. 2. **44 Ferramentas de Precisão no Cinto dos Mestres (As Micro-Ferramentas / Skills)**: Lupas, réguas, tesouras mágicas e parafusadeiras que realizam tarefas hiperespecíficas. 3. **65 Guardas Incorrutíveis na Porta (Os Quality Gates)**: Inspetores robóticos que não deixam passar nada se tiver um único parafuso torto ou se faltar a etiqueta de segurança. 4. **132 Peças Fundamentais (Os Componentes Agnósticos)**: Blocos de encaixe perfeito que funcionam em qualquer mesa de trabalho (seja no Claude, no Gemini, no Cursor, no OpenCode ou no Mimo).

Neste livro, nós vamos passear por cada canto dessa fábrica e tirar uma **FOTO** detalhada de cada peça!

PARTE I: AS 8 FERRAMENTAS MACRO (AS GRANDES OFICINAS)

Como ler as fichas desta parte: 11 dimensões antigas, 15 dimensões atuais

As fichas das 8 ferramentas são o registro da auditoria de 2026-09-22, feita antes de o ecossistema adotar a **Lente 15-D**. Elas usam uma matriz de **11 dimensões**. O padrão atual tem **15** (D1 a D15, lista completa na Parte V). A tabela mostra onde cada dimensão antiga cai na lente atual:

Dimensão antiga (ficha de 2026-09-22)	Onde cai na Lente 15-D
1. Recebe (Input)	D2 Input e Gatilhos · D7 O que o Estágio Recebe
2. Cria / Processa	D6 O que o Estágio Faz · D8 O que o Estágio Processa
3. Entrega (Output)	D9 O que o Estágio Entrega · D15 Output Consolidado e Handoff
4. Configurações	D2 Input e Gatilhos
5. Guardas (Gates)	D13 Quality Gates
6. Scripts determinísticos (0 LLM)	D8 O que o Estágio Processa
7. Hooks · 8. Agents · 9. Skills · 10. MCPs	D4 Componentes e Fractalidade
11. Rules (AGENTS.md)	D1 Contratos e Regras

Seis dimensões da Lente 15-D não existiam na matriz antiga: D3 Raio de Impacto e Isolamento, D5 Visão e Escopo, D10 Orquestração e Topologia, D11 Tratamento de Exceções e Fallback, D12 Observabilidade e Frugalidade e D14 Critério de Rejeição (Rollback). Nas 8 ferramentas elas continuam **sem avaliação** até cada uma passar pelo seu primeiro ciclo 15-D (`python ecossistema.py audit-4f --manifest <json>`).

Capítulo 1: AIDD Forge — O Altar da Fundação e os Guardiões das Leis

Na Festa (A Metáfora Memorável)

O Mestre Ferreiro que prepara o chão da oficina, coloca a bigorna, estende as regras na parede e tranca a porta para ninguém entrar bagunçando.

Antes de qualquer parede ser erguida, é este ferreiro que crava as estacas no chão e escreve, na própria parede da oficina, as regras que toda construção futura vai ter que obedecer. Sem terreno preparado e sem regra escrita, não existe planta nem construção — é por isso que ele entra primeiro, e é por isso que ele mesmo não ergue tijolo nenhum.

Na Casa (A Foto Técnica e a Ficha Histórica de 22/09/2026)

- **Caminho Físico no Disco:** `tools/aidd-forge`
- **Arquivo Canônico de Regras:** `tools/aidd-forge/AGENTS.md`

Nota de leitura: ficha histórica de 2026-09-22, na matriz antiga de **11 dimensões** (correspondência com a Lente 15-D no início desta parte); laudo 15-D: `docs/auditoria/aidd-forge/ciclo-01/`

LAUDO-15D-INICIAL.md, docs/auditoria/aidd-forge/ciclo-01/LAUDO-15D-REVISADO.md, docs/auditoria/aidd-forge/ciclo-02/LAUDO-15D-INICIAL.md, docs/auditoria/aidd-forge/ciclo-02/LAUDO-15D-REVISADO.md

Ficha de Auditoria Bit a Bit: [aidd-forge](#)

Ferramenta: [tools/aidd-forge](#)
Data da Auditoria: 2026-09-22
Status: AUDITADO / ÍNTEGRO
Responsável: Antigravity / Eco-AIDD Agent

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)

1. Matriz Contratual das 11 Dimensões

Dimensão	Detalhamento Técnico no Código
1. RECEBE (Input)	CLI Click (<code>init</code> , <code>inject</code> , <code>audit</code> , <code>conform</code>); payloads validados por <code>injection_request.schema.json</code> ; caminho do projeto alvo (<code>path: str</code>).
2. CRIA / PROCESSA	<code>Injector</code> (árvore estrutural), <code>PhaseFencer</code> (10 fases isoladas), <code>SlashRouter</code> (roteador de slash commands agnósticos), <code>UniversalInjector</code> + <code>materializador.py</code> (injeção atômica com rollback transacional), <code>ConformEngine</code> (autocura).
3. ENTREGA (Output)	Árvore de governança (<code>.agent/</code> , <code>.claude/</code> , <code>.gemini/</code> , <code>.cursor/</code> , <code>.windsurf/</code>), scripts de qualidade em <code>gates/</code> , relatórios auditáveis (Markdown, JSON, HTML) e hook <code>.git/hooks/pre-commit</code> .
4. CONFIGURAÇÕES (Configs)	Flags CLI <code>--force</code> , <code>--dry-run</code> , <code>--format json\ md\ html</code> , <code>--output</code> , <code>--item</code> ; mapeamento de aliases de IDE em <code>IDE_RULE_ALIASES</code> .
5. GUARDAS (Gates)	12 quality gates em <code>templates/gates/</code> : <code>G_BLOQUEAR_SEGREDOS</code> , <code>G_CONTRACTS</code> , <code>G_CYBERSECURITY_OWASP</code> , <code>G_DETERMINISMO_LEI_1</code> , <code>G ESTRUTURA_AST</code> , <code>G_HARNESS_COMPAT</code> , <code>G_INJECT</code> , <code>G_PERFORMANCE</code> , <code>G_QUARTETO_SINE_QUA_NON</code> , <code>G_SAIDA_BINARIA</code> , <code>G_STACK_PADRAO_OURO</code> , <code>G_TESTES_REAIS</code> .
6. SCRIPTS DETERMINÍSTICOS (0 LLM)	100% de <code>aidd-forge/core/</code> implementado com AST Python, regex compilado, manipulação atômica de disco e padrão <code>Result Monad</code> (<code>Result.ok</code> , <code>Result.fail</code>).
7. CAMPAINHAS DE ALERTA (Hooks)	<code>GitHooksInstaller</code> instala <code>.git/hooks/pre-commit</code> com execução compulsória dos gates antes de qualquer commit git.
8. PESSOAS / PERSONAS (Agents)	<code>subagent_purger.py</code> garante que subagentes cognitivos sejam estritamente efêmeros e expurgados após validação.
9. TAREFAS ÚNICAS (Skills)	10 skills embutidas: <code>aidd-grill</code> , <code>aidd-spec</code> , <code>aidd-tdd</code> , <code>aidd-tickets</code> , <code>caveman-ultra</code> , <code>cybersecurity-first</code> , <code>impeccable-ui</code> , <code>open-code-review-graph</code> , <code>orca-orchestrator</code> , <code>post-mortem</code> . Skill de orquestração externa: <code>aidd-forge-runner</code> .

Dimensão	Detalhamento Técnico no Código
10. TELEFONES EXTERNOS (MCPs)	Ponto de integração nativo com code-review-graph e suporte a injeção declarativa via forge inject mcp .
11. BILHETES (Rules / AGENTS.md)	Diretrizes em tools/aidd-forge/AGENTS.md : Zero Stubs, Result Monad compulsório, Context-Purge Isolation e Universal Injector.

2. Evidência de Testes e Portões

- **Comando executado:** `pytest tools/aidd-forge/tests`
- **Resultado:** 294 testes passaram, 0 falhas, 1 pulado (tempo de execução: 26.84s).
- **Invariantes testadas:** Rollback em falha de escrita, ancoragem em [AGENTS.md](#), isolamento de harnesses, detecção de camadas arquiteturais.

3. Diagnóstico de Não-Conformidades

- Nenhum bug funcional impeditivo encontrado no núcleo do [aidd-forge](#).
- Observação: Garantir que novos quality gates adicionados ao ecossistema raiz sejam sincronizados dinamicamente na pasta `templates/gates/` do [aidd-forge](#).

SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA

(Nenhum ticket impeditivo aberto para esta ferramenta no momento. Estado funcional 100% íntegro).

SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)

- **Status do Módulo:** APROVADO & CERTIFICADO
- **Nível de Autonomia:** 100% Determinístico (0 LLM no núcleo)
- **Handoff:** Registrado no [manifesto_auditoria.json](#). Próximo módulo liberado: [aidd-planner](#).

Capítulo 2: AIDD Planner — A Mesa do Arquiteto e o Livro de Receitas

Na Festa (A Metáfora Memorável)

O Grande Arquiteto que escuta o sonho do cliente, desenha a planta baixa em papel milimetrado com todas as medidas, cores e quartos, e entrega uma receita que qualquer cozinheiro consegue seguir.

Com o terreno pronto e as regras na parede, é aqui que o sonho do cliente vira desenho técnico: cada cômodo, medida, porta e contrato descritos em detalhe antes de qualquer prego ser batido. O arquiteto entrega a planta — quem constrói é o próximo mestre da linha.

Na Casa (A Foto Técnica e a Ficha Histórica de 22/09/2026)

- **Caminho Físico no Disco:** `tools/aidd-planner`
- **Arquivo Canônico de Regras:** `tools/aidd-planner/AGENTS.md`

Nota de leitura: ficha histórica de 2026-09-22, na matriz antiga de **11 dimensões** (correspondência com a Lente 15-D no início desta parte); esta ferramenta ainda não passou por um ciclo 15-D.

Ficha de Auditoria Bit a Bit: aidd-planner

Ferramenta: tools/aidd-planner
Data da Auditoria: 2026-09-22
Status: AUDITADO / ÍNTEGRO
Responsável: Antigravity / Eco-AIDD Agent

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)

1. Matriz Contratual das 11 Dimensões

Dimensão	Detalhamento Técnico no Código
1. RECEBE (Input)	CLI argparse (init, validate, export, audit). Parâmetros: --fluxo <1\ 2\ 3>, --nome, --slug, --descricao, --dominio, --pasta, --arquivo, --formato <factory\ pipeline\ dispatch>, --saida.
2. CRIA / PROCESSA	planner_engine.py: gerar_template_plano, validar_plano (JSON Schema + regras semânticas), exportar_para_fluxo_factory, exportar_para_pipeline_execucao, compilar_grafo_topologico_vsa. design_system.py: geração determinística (hash do projeto) de paleta cromática e identidade visual única.
3. ENTREGA (Output)	Arquivos: PLANNER.json, DESIGN-SYSTEM.json, HANDOFF_PLANNER_ENGINE.json, artefatos exportados para pipeline/dispatch.
4. CONFIGURAÇÕES (Configs)	Mapeamento MAPA_FLUXOS suportando aliases numéricos e textuais (1, 2, 3, pure, open, freedom). Flags --force, --dry-run.
5. GUARDAS (Gates)	Validação integrada contra schemas/planner_schema.json, verificação obrigatória do Quarteto Sine Qua Non (/docs, /webhooks, /mcp, /docs/guia), BDD Gherkin obrigatório, detecção de ciclos em grafo de dependências VSA.
6. SCRIPTS DETERMINÍSTICOS (0 LLM)	100% determinístico. Cálculos de hash para paleta cromática, validação estrita JSON Schema, algoritmo topológico de Kahn para ordenação DAG de fatias VSA.
7. CAMPAINHAS DE ALERTA (Hooks)	Emite códigos de saída binários (exit 0 sucesso, exit 1 falha) e relatórios formatados em stderr para interceptação por orquestradores superiores.
8. PESSOAS / PERSONAS (Agents)	Atua como motor de intake alimentando personas dos geradores especializados (aidd-pure, aidd-open, aidd-freedom).
9. TAREFAS ÚNICAS (Skills)	Skills associadas no ecossistema: aidd-planner-runner, aidd-plan, aidd-planos.
10. TELEFONES EXTERNOS (MCPs)	Totalmente desacoplado e autônomo (não necessita de MCP externo para execução de intake e exportação).

Dimensão	Detalhamento Técnico no Código
11. BILHETES (Rules / AGENTS.md)	<code>tools/aidd-planner/AGENTS.md</code> fixa: Invariante Zero Stubs, Quarteto Sine Qua Non ativado por default, envelope estrito aidd-ops para export factory.

2. Evidência de Testes e Portões

- **Comando executado:** `pytest tools/aidd-planner/tests`
- **Resultado:** 24 testes passaram, 0 falhas (tempo de execução: 1.25s).
- **Invariantes testadas:** Rejeição de plano sem quarteto, rejeição de plano sem BDD, exportação para factory envelope ops, compilação de grafo VSA com detecção de ciclo.

3. Diagnóstico de Não-Conformidades

- Nenhum bug impeditivo. Conformidade 100% comprovada na geração de envelopes de handoff formal e compatibilidade com VSA.

SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA

(Nenhum ticket impeditivo aberto. Módulo plenamente funcional).

SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)

- **Status do Módulo:** APROVADO & CERTIFICADO
- **Nível de Autonomia:** 100% Determinístico
- **Handoff:** Registrado no `manifesto_auditoria.json`. Próximo módulo: `aidd-generator` (Fluxo 01).

Capítulo 3: AIDD Generator — O Trem Autônomo de 8 Vagões (Fluxo 01: Do Zero Puro)

Na Festa (A Metáfora Memorável)

Um trem mágico com 8 vagões sequenciais. Ele recebe a ideia pura em uma ponta e, vagão por vagão (pesquisa, desenho, teste, programação), entrega uma cidade inteira de brinquedo montada e funcionando.

Com a planta em mãos, este é o motor que efetivamente ergue a construção do zero: vagão por vagão, a ideia pura vira aplicação funcionando, sem depender de nenhum tijolo pré-fabricado de terceiros.

Na Casa (A Foto Técnica e a Ficha Histórica de 22/09/2026)

- **Caminho Físico no Disco:** `tools/aidd-pure`
- **Arquivo Canônico de Regras:** `tools/aidd-pure/AGENTS.md`

Nota de leitura: ficha histórica de 2026-09-22, na matriz antiga de **11 dimensões** (correspondência com a Lente 15-D no início desta parte); esta ferramenta ainda não passou por um ciclo 15-D.

Ficha de Auditoria Bit a Bit: [aidd-generator](#) (Fluxo 01 — Do Zero Puro)

Ferramenta: [tools/aidd-generator](#)
Data da Auditoria: 2026-09-22
Status: AUDITADO / ÍNTEGRO
Responsável: Antigravity / Eco-AIDD Agent

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)

1. Matriz Contratual das 11 Dimensões

Dimensão	Detalhamento Técnico no Código
1. RECEBE (Input)	Ideia textual ou plano via CLI (<code>python scripts/pipeline_completo.py "<ideia>"</code>), flags <code>--modo <eco fast full></code> , <code>--verbose</code> , <code>--skip-fase <N></code> , <code>--retomar</code> , <code>--plano <path.json></code> . Consome contrato de handoff do planner (<code>HANDOFF_PLANNER_ENGINE.json</code>).
2. CRIA / PROCESSA	Pipeline autônoma de 8 fases sequenciais orquestrada por Máquina de Estados Finitos (<code>fsm_engine.py</code>): Fase 1 (Pesquisa), Fase 2 (Análise de Requisitos), Fase 3 (Design Arquitetural & UI), Fase 4 (Planejador de Slices), Fase 5 (Criador de Código/TDD Red), Fase 6 (Documentador & OpenAPI), Fase 7 (Auto-Crítica & Auditoria), Fase 8 (Implementador Green & Refactor).
3. ENTREGA (Output)	Código-fonte completo do projeto (Frontend Next.js + TS + Tailwind, Backend Python puro VSA + SQLite WAL), <code>PLANO-EXECUCAO-ESTRUTURADO.json</code> , testes automatizados, relatórios de auditoria e tokenomics (<code>benchmark_tokenomics.py</code>).
4. CONFIGURAÇÕES (Configs)	Modos de operação: <code>eco</code> (mínimo de tokens), <code>fast</code> (direto ao ponto), <code>full</code> (máxima profundidade). Configuração de orquestração via Prefect (<code>pipeline_prefect.py</code>) ou fallback nativo Python.
5. GUARDAS (Gates)	Validação de transição entre cada uma das 8 fases em <code>scripts/gates/</code> e <code>verificar_gates.py</code> . Rejeição binária imediata se uma fase não atingir 100% dos requisitos contratuais.
6. SCRIPTS DETERMINÍSTICOS (O LLM)	Mecanismo FSM (<code>fsm_engine.py</code>), validação de esquemas de artefatos por JSON Schema Draft 2020-12, checagem de integridade de arquivos gerados e telemetria determinística de tokens.
7. CAMPAINHAS DE ALERTA (Hooks)	Emissão de alertas de quebra de contrato de fase e interrupção compulsória com preservação de estado transacional.
8. PESSOAS / PERSONAS (Agents)	8 personas operacionais especializadas em cada fase (Pesquisador, Analista, Designer, Planejador, Criador, Documentador, Crítico, Implementador).

Dimensão	Detalhamento Técnico no Código
9. TAREFAS ÚNICAS (Skills)	Skills integradas: aidd-generator-runner , aidd-pure , fluxo-01-runner , aidd-spec , aidd-tickets , aidd-tdd .
10. TELEFONES EXTERNOS (MCPs)	Suporte a chamadas de inspeção estática via code-review-graph e preflight checks de ambiente.
11. BILHETES (Rules / AGENTS.md)	tools/aidd-generator/AGENTS.md : Persistência de estado exclusivamente em JSON, transições mecânicas estritas, Zero Stubs na Fase 8.

2. Evidência de Testes e Portões

- **Comando executado:** `pytest tools/aidd-generator/tests`
- **Resultado:** 1006 testes passaram, 0 falhas, 5 pulados (Prefect opcional) em 26.50s.
- **Invariantes testadas:** Transparência de tokens, fencer de isolamento de fases, microtarefas da Fase 8, roteamento de slash commands e validação de gates.

3. Diagnóstico de Não-Conformidades

- Nenhum bug impeditivo. 100% de integridade comprovada nos 1006 testes unitários e de integração.

SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA

(Nenhum ticket impeditivo aberto. Módulo plenamente funcional).

SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)

- **Status do Módulo:** APROVADO & CERTIFICADO
- **Nível de Autonomia:** Pipeline autônoma de 8 fases com barreira determinística.
- **Handoff:** Registrado no [manifesto_auditoria.json](#). Próximo módulo: [aidd-factory](#) (Fluxo 02).

Capítulo 4: AIDD Factory — A Linha de Montagem de Peças Prontas (Fluxo 02: Open-Source)

Na Festa (A Metáfora Memorável)

O Mestre Montador que pega os melhores motores de brinquedo já inventados no mundo (motores abertos) e os conecta perfeitamente com cabos fortes para criar um veículo superpotente sem reinventar a roda.

Também constrói a partir da mesma planta, mas em vez de erguer tijolo por tijolo, monta a casa com módulos open-source já prontos, testados por milhares de outras obras e encaixados sob medida no projeto.

Na Casa (A Foto Técnica e a Ficha Histórica de 22/09/2026)

- **Caminho Físico no Disco:** `tools/aidd-open`
- **Arquivo Canônico de Regras:** `tools/aidd-open/AGENTS.md`

Nota de leitura: ficha histórica de 2026-09-22, na matriz antiga de **11 dimensões** (correspondência com a Lente 15-D no início desta parte); esta ferramenta ainda não passou por um ciclo 15-D.

Ficha de Auditoria Bit a Bit: aidd-factory (Fluxo 02 — Motores Open-Source)

Ferramenta: `tools/aidd-factory`
Data da Auditoria: 2026-09-22
Status: AUDITADO / ÍNTEGRO
Responsável: Antigravity / Eco-AIDD Agent

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)

1. Matriz Contratual das 11 Dimensões

Dimensão	Detalhamento Técnico no Código
1. RECEBE (Input)	Consome estritamente <code>PLANO-INFRAESTRUTURA.json</code> em conformidade com <code>componentes/compartilhado/specs/plano-infraestrutura.schema.json</code> gerado pelo intake do <code>aidd-planner</code> ou <code>aidd-ops</code> . CLI: <code>python scripts/pipeline_factory.py --plano <path.json> --destino <dir></code> .
2. CRIA / PROCESSA	Pipeline de integração de serviços open-source em fatias VSA: Fase 1 (<code>01_analisador.py</code> — análise de nicho e desdobramento em blocos), Fase 4 (<code>04_compose.py</code> — geração determinística de <code>docker-compose.yml</code>), Fase 5 (<code>05_init_db.py</code> — scripts SQL de inicialização multi-tenant/bancos lógicos), Fase 6 (<code>06_env.py</code> — variáveis de ambiente seguras com fallbacks tipados), Fase 9 (<code>09_integracao.py</code> — gateway de integração VSA e Quarteto Sine Qua Non).
3. ENTREGA (Output)	Contrato canônico <code>FACTORY_OUTPUT.json</code> , manifesto de serviços, compose funcional, scripts de banco de dados, gateway FastAPI VSA e stack frontend Next.js + TS + Tailwind.
4. CONFIGURAÇÕES (Configs)	Parâmetros de CLI (<code>--plano</code> , <code>--destino</code> , <code>--dry-run</code> , <code>--skip-frontend</code>). Suporte a nichos de catálogo fixo (<code>clinicas</code> , <code>delivery</code> , <code>farmacias</code> , <code>b2b_industrial</code> , <code>energia_solar</code>) e nicho dinâmico (<code>dinamico_<slug></code>).
5. GUARDAS (Gates)	<code>G_FACTORY_ANALYSIS</code> , <code>G_FACTORY_COMPOSE</code> , <code>G_FACTORY_ENV</code> , <code>G_FACTORY_INIT_DB</code> , <code>G_FACTORY_INTEGRATION</code> e <code>G_FACTORY_MVP</code> (em <code>tools/aidd-open/gates/</code>). <code>G_FACTORY_INPUT</code> , <code>G_FACTORY_OUTPUT</code> e <code>G_FACTORY_DETERMINISTIC</code> são rótulos de invariante no <code>AGENTS.md</code> , não arquivos de portão.
6. SCRIPTS DETERMINÍSTICOS (0 LLM)	100% determinístico nas fases estruturais: análise de topologia, geração do Docker Compose com Traefik, particionamento de bancos lógicos PostgreSQL e geração de <code>.env</code> .
7. CAMPAINHAS DE ALERTA (Hooks)	Validações em tempo de execução via <code>contrato_factory.py</code> emitindo interrupção com traceback rastreável ao detectar ausência de dependências.

Dimensão	Detalhamento Técnico no Código
8. PESSOAS / PERSONAS (Agents)	Agente orquestrador de fábrica open-source e fatia- dor de integração VSA.
9. TAREFAS ÚNICAS (Skills)	Skills associadas: <code>aidd-factory-runner</code> , <code>aidd-open</code> , <code>open</code> , <code>fluxo-02-runner</code> .
10. TELEFONES EXTERNOS (MCPs)	Totalmente autônomo na orquestração de templates e fatias de código.
11. BILHETES (Rules / AGENTS.md)	<code>tools/aidd-factory/AGENTS.md</code> : Consumo exclusivo do envelope de infraestrutura canônico, Zero Stubs, reuso compulsório de <code>result.py</code> e <code>escritor_atomico</code> . <code>py</code> .

2. Evidência de Testes e Portões

- **Comando executado:** `pytest tools/aidd-factory/tests`
- **Resultado:** 19 testes passaram, 0 falhas em 94.95s.
- **Invariantes testadas:** Validação de planos de clínicas e delivery, compilação de compose e init_db, gateway generator, frontend factory real compilando sem erros e suporte a nicho dinâmico sem LLM.

3. Diagnóstico de Não-Conformidades

- Nenhum bug impeditivo. Módulo atende plenamente à governança e à Lei #11.

SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA

(Nenhum ticket impeditivo aberto. Módulo plenamente funcional).

SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)

- **Status do Módulo:** APROVADO & CERTIFICADO
- **Nível de Autonomia:** Pipeline de integração determinística multi-serviços com validação real de compilação.
- **Handoff:** Registrado no `manifesto_auditoria.json`. Próximo módulo: `aidd-bridge` (Fluxo 03).

Capítulo 5: AIDD Bridge — A Ponte da Libertação (Fluxo 03: Desacoplamento Low-Code)

Na Festa (A Metáfora Memorável)

O Chaveiro Libertador que resgata os brinquedos que estavam presos em gaiolas com cadeados caros de empresas distantes (Lovable, Supabase), limpando-os para funcionarem livres no quintal da sua própria casa.

Aqui a casa já existe — só que presa a um dono que cobra aluguel para você nem abrir a porta. Este é o chaveiro que destranca as paredes de vendor lock-in e devolve as chaves de verdade para o dono real do código.

Na Casa (A Foto Técnica e a Ficha Histórica de 22/09/2026)

- **Caminho Físico no Disco:** [tools/aidd-freedom](#)
- **Arquivo Canônico de Regras:** [tools/aidd-freedom/AGENTS.md](#)

Nota de leitura: ficha histórica de 2026-09-22, na matriz antiga de **11 dimensões** (correspondência com a Lente 15-D no início desta parte); laudo 15-D: docs/auditoria/aidd-freedom/ciclo-02/LAUDO-15D-INICIAL.md, docs/auditoria/aidd-freedom/ciclo-02/LAUDO-15D-REVISADO.md

Ficha de Auditoria Bit a Bit: [aidd-bridge](#) (Fluxo 03 — Low-Code / Desacoplamento)

Ferramenta: [tools/aidd-bridge](#)
Data da Auditoria: 2026-09-22
Status: AUDITADO / ÍNTEGRO
Responsável: Antigravity / Eco-AIDD Agent

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)

1. Matriz Contratual das 11 Dimensões

Dimensão	Detalhamento Técnico no Código
1. RECEBE (Input)	CLI argparse: subcomandos scan , convert-db , merge , pack , migrate-auth , destroy , unpack . Consome projetos exportados do Lovable/v0/Bolt contendo React + Vite + Tailwind + migrações Supabase.
2. CRIA / PROCESSA	scanner.py (análise de páginas, rotas e migrações), data_bridge.py + sql_transpiler.py (sanitização de SQL Supabase para PostgreSQL padrão), unifier.py (agregação multi-app em fatias VSA), devops.py (geração de Dockerfile, Compose e Swarm), auth_migrator.py (migração de hashes de senha de auth.users), frontend_liberator.py (remoção de bibliotecas proprietárias), vsa_exporter.py (exportação do Quarteto Sine Qua Non dinâmico), pipeline_bridge.py (orquestração 6 fases).
3. ENTREGA (Output)	Aplicação 100% livre de lock-in, banco PostgreSQL consolidado (init-db.sql), stack Docker Swarm pronta para VPS, fatias verticais VSA em src/modules/ e diretório quarteto_sine_qua_non/ contendo Swagger, Webhook, MCP e Guia de Uso.
4. CONFIGURAÇÕES (Configs)	Flags CLI <code>--stack <lite\ full></code> , <code>--domain</code> , <code>--output</code> , <code>--traefik-network</code> , <code>--certresolver</code> , <code>--apply</code> , <code>--yes</code> , <code>--with-gottrue</code> .
5. GUARDAS (Gates)	Quality gates dedicados em gates/ : G_BRIDGE_VENDOR_LOCKIN , G_BRIDGE_DOCKER_OCI , G_BRIDGE_POSTGRESQL , G_BRIDGE_VSA_COMPAT .
6. SCRIPTS DETERMINÍSTICOS (O LLM)	100% determinístico. Transpilação SQL por regex e gramática estática, manipulação de AST TypeScript/JavaScript para substituição de importações do cliente @supabase/supabase-js e empacotamento de templates Docker.
7. CAMPAINHAS DE ALERTA (Hooks)	_forcar_utf8_stdio para compatibilidade multi-plataforma (Windows cp1252), verificações de credenciais obrigatórias para ações remotas em VPS / Cloudflare DNS e prompt interativo obrigatório no comando destrutivo destroy .

Dimensão	Detalhamento Técnico no Código
8. PESSOAS / PERSONAS (Agents)	Agente especialista em desmonte de lock-in e unificação de apps legadas.
9. TAREFAS ÚNICAS (Skills)	Skills associadas: aidd-bridge-runner , aidd-freedom , freedom , fluxo-03-runner .
10. TELEFONES EXTERNOS (MCPs)	Módulo de automação de DNS com Cloudflare API (cloudflare_dns.py) e orquestração SSH para provisionamento de VPS.
11. BILHETES (Rules / AGENTS.md)	tools/aidd-bridge/AGENTS.md : Desacoplamento estrito, migração segura de credenciais, rollback em teardown e Quarteto Sine Qua Non mandatário.

2. Evidência de Testes e Portões

- **Comando executado:** `pytest tools/aidd-bridge/tests`
- **Resultado:** 64 testes passaram, 0 falhas em 1.19s.
- **Invariantes testadas:** Transpilação SQL Supabase → PostgreSQL, empacotamento de stack Swarm, unificação de fatias VSA, migração de hashes de auth e pipeline e2e do Fluxo 03.

3. Diagnóstico de Não-Conformidades

- Nenhum bug impeditivo. Módulo atende 100% às exigências do ecossistema.

SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA

(Nenhum ticket impeditivo aberto. Módulo plenamente funcional).

SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)

- **Status do Módulo:** APROVADO & CERTIFICADO
- **Nível de Autonomia:** Pipeline determinístico de 6 fases com extração limpa e zero dependência de IA para sanitização.
- **Handoff:** Registrado no [manifesto_auditoria.json](#). Próximo módulo: [aidd-master](#) (Meso-camada e convergência).

Capítulo 6: AIDD Master — O Maestro da Harmonização Monolítica VSA

Na Festa (A Metáfora Memorável)

O Maestro da Orquestra que garante que todos os instrumentos (fatias verticais) toquem em harmonia perfeita no mesmo salão, sem um bater no outro e com uma barreira invisível de segurança.

Com as construções de pé, é este maestro que garante que todas as alas do prédio funcionem como um único edifício coerente — nenhuma fatia vertical pisa no cano ou na fiação da vizinha.

Na Casa (A Foto Técnica e a Ficha Histórica de 22/09/2026)

- **Caminho Físico no Disco:** `tools/aidd-master`
- **Arquivo Canônico de Regras:** `tools/aidd-master/AGENTS.md`

Nota de leitura: ficha histórica de 2026-09-22, na matriz antiga de **11 dimensões** (correspondência com a Lente 15-D no início desta parte); esta ferramenta ainda não passou por um ciclo 15-D.

Ficha de Auditoria Bit a Bit: aidd-master (Meso-Camada de Convergência Monolítica VSA)

Ferramenta: tools/aidd-master
Data da Auditoria: 2026-09-22
Status: AUDITADO / ÍNTEGRO
Responsável: Antigravity / Eco-AIDD Agent

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)

1. Matriz Contratual das 11 Dimensões

Dimensão	Detalhamento Técnico no Código
1. RECEBE (Input)	Comandos CLI: <code>python scripts/aidd.py [add-module compose-orca run-all provision], dispatch_pipeline.py --planner <path.json> \ -dispatch <json></code> , manifestos de fatias VSA e contratos de handoff das três trilhas da Tríade.
2. CRIA / PROCESSA	Orquestração da Meso-Camada VSA: <code>dispatch_pipeline.py</code> (Kahn DAG para ordenação topológica de fatias), <code>engine_router.py</code> (despacho para o motor correto 01/02/03), <code>vsa_join_barrier.py</code> (barreira de validação síncrona com Git Worktrees efêmeras e merge na master), <code>orchestrator_pipeline.py</code> (orquestrador mestre), <code>add_module.py</code> (scaffolding de fatias VSA verticais independentes: router, service, repository, dtos, events).
3. ENTREGA (Output)	Monólito Modular VSA consolidado com SQLite WAL otimizado, barreira transacional unificada, contratos OpenAPI 3.1 tipados e convertidos para TypeScript (<code>openapi_to_ts.py</code>) e Quarteto Sine Qua Non dinâmico.
4. CONFIGURAÇÕES (Configs)	Configurações de pool SQLite WAL (<code>PRAGMA journal_mode=WAL</code>), limites de concorrência e barreira de tolerância a falhas (<code>--timeout</code> , <code>--retry</code> , <code>--fail-fast</code>).
5. GUARDAS (Gates)	12 quality gates determinísticos em <code>scripts/gates/</code> : <code>G_ARQUITETURA</code> , <code>G_AST_BOUNDED_CONTEXT</code> , <code>G_CHAOS</code> , <code>G_CONTRACTS</code> , <code>G_ESTRUTURA</code> , <code>G_HARNESS_COMPAT</code> , <code>G_INJECT</code> , <code>G_PERFORMANCE</code> , <code>G_QUALIDADE</code> , <code>G_SEGREDOS</code> , <code>G_SEGURANCA</code> , <code>G_TESTES</code> .
6. SCRIPTS DETERMINÍSTICOS (0 LLM)	100% determinístico. Barreira de junção VSA baseada em Git Worktrees, ordenação topológica por algoritmo de Kahn, AST linter para isolamento de contextos delimitados (proibição de importações cruzadas não autorizadas) e Result Monad.
7. CAMPAINHAS DE ALERTA (Hooks)	Monitoramento de latência e p99 (<code>test_live.py</code> , <code>G_PERFORMANCE</code>), detecção de lock e retry automático em SQLite (<code>sqlite_busy_retry.py</code>), e bloqueio binário em caso de divergência de schema na Join Barrier.

Dimensão	Detalhamento Técnico no Código
8. PESSOAS / PERSONAS (Agents)	Agente orquestrador de convergência e maestro de fatias verticais VSA.
9. TAREFAS ÚNICAS (Skills)	Skills associadas: <code>aidd-master-runner</code> , <code>aidd-dispatch-runner</code> , <code>aidd-pipeline-runner</code> , <code>aidd-orchestrator-runner</code> .
10. TELEFONES EXTERNOS (MCPs)	Suporte completo a MCP Server SDK (<code>test_mcp_server_sdk.py</code>) permitindo expor as fatias do monólito como ferramentas MCP padronizadas.
11. BILHETES (Rules / AGENTS.md)	<code>tools/aidd-master/AGENTS.md</code> : Isolamento estrito de bounded contexts, Result Monad compulsório, SQLite WAL e proibição categórica de stubs.

2. Evidência de Testes e Portões

- **Comando executado:** `pytest tools/aidd-master/tests`
- **Resultado:** 439 testes passaram, 0 falhas, 3 pulados em 266.59s (04:26 min).
- **Invariantes testadas:** Bounded context AST isolation, VSA Join Barrier, outbox worker, SQLite busy retry, JWT hardening, Next.js exporter e validação dos 12 quality gates.

3. Diagnóstico de Não-Conformidades

- Nenhum bug impeditivo. Módulo é a coluna vertebral de convergência da Tríade e opera com máxima estabilidade.

SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA

(Nenhum ticket impeditivo aberto. Módulo plenamente funcional).

SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)

- **Status do Módulo:** APROVADO & CERTIFICADO
- **Nível de Autonomia:** Meso-camada determinística com barreira formal de sincronização e merge limpo de fatias VSA.
- **Handoff:** Registrado no `manifesto_auditoria.json`. Próximo módulo: `aidd-enterprise`.

Capítulo 7: AIDD Enterprise — O Cofre de Alta Segurança e Selos Criptográficos

Na Festa (A Metáfora Memorável)

O Inspetor do Cofre do Rei que confere o carimbo de ouro (assinatura digital SHA-256) em cada documento e garante que nenhum espião consiga alterar uma única linha de código.

É o inspetor que sela cada ambiente com um lacre inviolável (assinatura criptográfica) antes de qualquer chave ser entregue para o mundo real — se o lacre estiver quebrado, a entrega não sai.

Na Casa (A Foto Técnica e a Ficha Histórica de 22/09/2026)

- **Caminho Físico no Disco:** `tools/aidd-enterprise`

- **Arquivo Canônico de Regras:** [tools/aidd-enterprise/AGENTS.md](#)

Nota de leitura: ficha histórica de 2026-09-22, na matriz antiga de **11 dimensões** (correspondência com a Lente 15-D no início desta parte); laudo 15-D: docs/auditoria/aidd-enterprise/ciclo-01/LAUDO-15D-INICIAL.md, docs/auditoria/aidd-enterprise/ciclo-01/LAUDO-15D-REVISADO.md

Ficha de Auditoria Bit a Bit: [aidd-enterprise](#) (Resiliência Crítica SHA-256)

Ferramenta: [tools/aidd-enterprise](#)
Data da Auditoria: 2026-09-22
Status: AUDITADO / ÍNTEGRO
Responsável: Antigravity / Eco-AIDD Agent

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)

1. Matriz Contratual das 11 Dimensões

Dimensão	Detalhamento Técnico no Código
1. RECEBE (Input)	Comandos CLI: <code>python scripts/aidd.py [add-module compose-orca run-all provision]</code> , componentes injetáveis (<code>scripts/injector/</code>) com validação via <code>component_manifest.json</code> e assinatura SHA-256.
2. CRIA / PROCESSA	Validação criptográfica de integridade de componentes, arquitetura Zero-Trust, isolamento de bounded contexts regulados, driver de eventos (<code>events_driver.py</code>), outbox worker transacional com idempotência, autenticação reforçada (JWT hardening, OIDC SSO, RLS).
3. ENTREGA (Output)	Monólito modular enterprise auditado, schemas OpenAPI 3.1 com tipagem TypeScript estrita, SQLite WAL com política de retenção e recovery, e trilhas de auditoria criptografadas.
4. CONFIGURAÇÕES (Configs)	Políticas de CSP (<code>security_csp_lib.py</code>), pool de conexões SQLite com retry determinístico (<code>sqlite_busy_retry.py</code>), limites de LRU de transações (<code>transaction_log_lru.py</code>).
5. GUARDAS (Gates)	10 quality gates locais em <code>scripts/gates/</code> : <code>G_ARQUITETURA</code> , <code>G_CHAOS</code> , <code>G_CONTRACTS</code> , <code>G ESTRUTURA</code> , <code>G_HARNESS_COMPAT</code> , <code>G_INJECT</code> , <code>G_PERFORMANCE</code> , <code>G_QUALIDADE</code> , <code>G_SEGREDOS</code> , <code>G_SEGURANCA</code> , <code>G_TESTES</code> .
6. SCRIPTS DETERMINÍSTICOS (0 LLM)	100% determinístico. Validação de hashes SHA-256, checagem AST de isolamento de bounded contexts, injeção de schemas sem dependência de inferência de IA e barreira Result Monad.
7. CAMPAINHAS DE ALERTA (Hooks)	<code>G_CHAOS</code> (injeção controlada de falhas para teste de resiliência), <code>G_SEGREDOS</code> (bloqueio de vazamento de credenciais) e <code>G_SEGURANCA</code> (defesa contra XSS/injeção SQL).
8. PESSOAS / PERSONAS (Agents)	Agente de conformidade enterprise e auditor de integridade criptográfica.
9. TAREFAS ÚNICAS (Skills)	Skills associadas: <code>aidd-enterprise-runner</code> , <code>aidd-diagnose</code> .

Dimensão	Detalhamento Técnico no Código
10. TELEFONES EXTERNOS (MCPs)	MCP Server SDK corporativo (test_mcp_server_sdk.py) e adaptadores de telemetria externa (OpenTelemetry , metrics.py).
11. BILHETES (Rules / AGENTS.md)	tools/aidd-enterprise/AGENTS.md : Integridade criptográfica compulsória, Result Monad em toda a camada de serviço, SQLite WAL e zero stubs.

2. Evidência de Testes e Portões

- **Comando executado:** `pytest tools/aidd-enterprise/tests`
- **Resultado:** 365 testes passaram, 0 falhas, 3 pulados em 148.87s (02:28 min).
- **Invariantes testadas:** Validação SHA-256 de manifestos, outbox idempotente, SSO OIDC, mitigação de XSS em webhooks, RLS e aprovação de todos os 10 quality gates.

3. Diagnóstico de Não-Conformidades

- Nenhum bug impeditivo. Módulo 100% íntegro e em estrita conformidade com as Leis do ecossistema.

SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA

(Nenhum ticket impeditivo aberto. Módulo plenamente funcional).

SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)

- **Status do Módulo:** APROVADO & CERTIFICADO
- **Nível de Autonomia:** Plataforma de missão crítica regulada 100% determinística.
- **Handoff:** Registrado no [manifesto_auditoria.json](#). Próximo módulo: [aidd-ops](#).

Capítulo 8: AIDD Ops — A Usina de Força e a Torre de Vigilância

Na Festa (A Metáfora Memorável)

O Chefe dos Engenheiros que constrói a usina de energia (servidor VPS), liga os motores (Docker), tranca as portas com chaves fortes (SSH seguro) e coloca câmeras de vigilância 24 horas por dia (Uptime Kuma).

É quem liga a energia do prédio já pronto, tranca as portas externas com fechaduras fortes e instala as câmeras de vigilância — para que a construção funcione 24 horas por dia sem ninguém arrombar a fechadura enquanto todos dormem.

Na Casa (A Foto Técnica e a Ficha Histórica de 22/09/2026)

- **Caminho Físico no Disco:** [tools/aidd-ops](#)
- **Arquivo Canônico de Regras:** [tools/aidd-ops/AGENTS.md](#)

Nota de leitura: ficha histórica de 2026-09-22, na matriz antiga de **11 dimensões** (correspondência com a Lente 15-D no início desta parte); esta ferramenta ainda não passou por um ciclo 15-D.

Ficha de Auditoria Bit a Bit: [aidd-ops](#) (Infraestrutura, VPS e Observabilidade)

Ferramenta: `tools/aidd-ops`
Data da Auditoria: 2026-09-22
Status: AUDITADO / ÍNTEGRO
Responsável: Antigravity / Eco-AIDD Agent

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)

1. Matriz Contratual das 11 Dimensões

Dimensão	Detalhamento Técnico no Código
1. RECEBE (Input)	CLI argparse: <code>python scripts/pipeline_ops.py [plan deploy monitor rotate-secrets]</code> . Parâmetros: <code>--texto</code> , <code>--nicho</code> , <code>--ferramentas-json</code> , <code>--dry-run</code> , <code>--vps-host</code> , <code>--vps-user</code> , <code>--ssh-key</code> .
2. CRIA / PROCESSA	Pipeline de provisionamento de infraestrutura em 3 fases de planejamento: Fase 1 (<code>01_intake.py</code> — detecção de nicho fixo ou dinâmico), Fase 2 (<code>02_curadoria.py</code> — seleção de stack open-source compatível), Fase 3 (<code>03_sizing.py</code> — dimensionamento de CPU/RAM/Disco por heurísticas determinísticas). Execução remota: <code>ssh_runner.py</code> (execução segura sem interpolação perigosa), <code>ansible</code> (hardening de SO e SSH via playbook idempotente), <code>uptime_kuma.py</code> (extração e exportação de probes reais de monitoramento), <code>rotate_secrets.py</code> (rotação de segredos via <code>sops</code> e <code>age</code>).
3. ENTREGA (Output)	<code>PLANO-INFRAESTRUTURA.json</code> em estrita conformidade com schema JSON, templates Docker Compose OCI, playbook Ansible executado, dashboard Uptime Kuma provisionado e credenciais criptografadas.
4. CONFIGURAÇÕES (Configs)	Parâmetros de CLI (<code>--dry-run</code> , <code>--ferramentas-json</code> , <code>-export-kuma</code>), catálogo de nichos (<code>data/catalogo_nichos.json</code>), requisitos de recursos (<code>data/requisitos_recursos.json</code>).
5. GUARDAS (Gates)	<code>G_INFRA_COMPOSE</code> (reprodutibilidade estrita dos manifests), <code>G_HADOLINT</code> (conformidade OCI dos Dockerfiles), <code>GateSshRunnerAST</code> (garantia de zero concatenação insegura em comandos remotos).
6. SCRIPTS DETERMINÍSTICOS (0 LLM)	100% determinístico. Dimensionamento de hardware por fórmulas lineares estáticas, parse e validação de compose via PyYAML, geração de playbooks e scripts de conexão SSH com Paramiko.
7. CAMPAINHAS DE ALERTA (Hooks)	Pré-voo SSH com Paramiko validando conectividade, autenticação e timeout antes de acionar playbooks; rejeição imediata se houver tentativa de injeção de parâmetros em comandos remotos.

Dimensão	Detalhamento Técnico no Código
8. PESSOAS / PERSONAS (Agents)	Agente orquestrador de operações de infraestrutura e observabilidade.
9. TAREFAS ÚNICAS (Skills)	Skills associadas: aidd-ops-runner , aidd-diagnose .
10. TELEFONES EXTERNOS (MCPs)	Módulos de conexão remota SSH (Paramiko/Ansible), Cloudflare DNS e integração com API do Uptime Kuma.
11. BILHETES (Rules / AGENTS.md)	tools/aidd-ops/AGENTS.md : Compose determinístico, observabilidade sem stubs, hardening via Ansible DevSec e suporte nativo ao envelope de nicho dinâmico.

2. Evidência de Testes e Portões

- **Comando executado:** `pytest tools/aidd-ops/tests`
- **Resultado:** 180 testes passaram, 0 falhas em 40.19s.
- **Invariantes testadas:** Hardening SSH/OS em dry-run, pré-voo Paramiko sem vazar credenciais, bloqueio AST de concatenação insegura, exportação e validação de monitores Uptime Kuma sem dados hardcoded.

3. Diagnóstico de Não-Conformidades

- Nenhum bug impeditivo. Módulo 100% íntegro e operacional.

SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA

(Nenhum ticket impeditivo aberto. Módulo plenamente funcional).

SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)

- **Status do Módulo:** APROVADO & CERTIFICADO
- **Nível de Autonomia:** Orquestrador determinístico de infraestrutura e observabilidade.
- **Handoff:** Registrado no [manifesto_auditoria.json](#). Todas as 8 ferramentas da pasta `tools/` auditadas com sucesso.

PARTE II: O CINTO DE UTILIDADES (AS 44 MICRO-FERRAMENTAS / SKILLS)

Na Festa

Imagine o cinto de utilidades do Batman ou a caixa de ferramentas mágica de um relojoeiro suíço. Cada uma dessas 44 ferramentas tem um formato único e resolve exatamente um problema específico sem fazer barulho nem desperdiçar energia.

Na Casa (O Catálogo Completo das 44 Skills Canônicas)

Localização canônica: [componentes/compartilhado/skills/](#)

Cada skill abaixo é lida pela **Lente 15-D**, o padrão de auditoria do ecossistema ([docs/auditoria/TEMPLATE-AUDITORIA-FERRAMENTA.md](#)): D1 a D4 governança e blindagem, D5 a D10 o trabalho em si, D11 e D12 resiliência e economia, D13 a D15 validação e entrega. O gerador preenche cada dimensão lendo o disco. Em D3, D11, D12, D14 e D15 ele diz só se o texto do [SKILL.md](#) **cita** o assunto: citar não é implementar, e nas skills de domínio externo a citação costuma ser sobre o produto ensinado (o [retry](#) da [agents-sdk](#) é o do SDK da Cloudflare). A prova de comportamento só existe onde há laudo 15-D de um ciclo de auditoria (Parte V).

1. Micro-Ferramenta: [aidd-9router](#)

- **Foto / Identidade:** [aidd-9router](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-9router/SKILL.md](#)
- **O que Faz (Missão Única):** Builds, calibrates, deploys and operates the house 9Router AI gateway end to end - token saver tuning, coding combos (code-pro, code-fast, code-free) chosen by real benchmarks, 24/7 deploy on the VPS with DNS, routing of Claude Code, OpenCode, MiMo and omp through the combos inside Orca ADE with an on/off switch, health diagnosis and cheap direct calls. Use when the user mentions 9Router, NINEROUTER_URL, NINEROUTER_KEY, combos, routing a harness or Orca agents through 9Router, deploying or checking the gateway, or which model to use for coding; also for image, TTS, STT, embeddings and web search/fetch through 9Router.
- **Lente 15-D desta Micro-Ferramenta:**
 - **D1. Contratos e Regras:** frontmatter [name/description](#) no [SKILL.md](#); distribuída pela fonte única [componentes/compartilhado/skills/aidd-9router/](#).
 - **D2. Input e Gatilhos:** gatilho pela [description](#) do frontmatter (nenhum arquivo em [comandos/](#) aponta para ela).
 - **D3. Raio de Impacto e Isolamento:** o [SKILL.md](#) não cita worktree nem isolamento.
 - **D4. Componentes e Fractalidade:** scripts: [_comum.py](#), [bench.py](#), [chamar.py](#), [deploy_vps.py](#), [doctor.py](#), [harness_9router.py](#), [instalar_wrappers.py](#), [omp_provider.py](#), [orca_9router.py](#) em [scripts/](#); MCPs citados: nenhum.
 - **D5. Visão e Escopo:** Builds, calibrates, deploys and operates the house 9Router AI gateway end to end - token saver tuning, coding combos (code-pro, code-fast, code-free) chosen by real benchmarks, 24/7 deploy on the VPS with DNS, routing of Claude Code, OpenCode, MiMo and omp through the combos inside Orca ADE with an on/off switch, health diagnosis and cheap direct calls. Use when the user mentions 9Router, NINEROUTER_URL, NINEROUTER_KEY, combos, routing a harness or Orca agents through 9Router, deploying

or checking the gateway, or which model to use for coding; also for image, TTS, STT, embeddings and web search/fetch through 9Router.

- ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md` e os scripts locais.
- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.
- ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`, `retry`.
- ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` cita `rollback`.
- ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita handoff. Ainda sem laudo 15-D próprio.

2. Micro-Ferramenta: `aidd-agent-writing`

- **Foto / Identidade:** `aidd-agent-writing`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-agent-writing/SKILL.md`
- **O que Faz (Missão Única):** Guides writing any document an agent consumes (context pointers, information hierarchy, completion criteria, pruning). Use when creating or editing a skill, or modifying `AGENTS.md` or `CLAUDE.md`, or the user says "escrever skill", "escrita para agentes".
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-agent-writing/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita `worktree` nem isolamento.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Guides writing any document an agent consumes (context pointers, information hierarchy, completion criteria, pruning). Use when creating or editing a skill, or modifying `AGENTS.md` or `CLAUDE.md`, or the user says "escrever skill", "escrita para agentes".
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md`.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
 - ▶ **D9. O que o Estágio Entrega:** o que o roteiro do `SKILL.md` orienta produzir (ver D5); não há motor próprio que grave arquivo.
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` não cita `log`, telemetria nem `secoes/`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita `rollback` nem limpeza.
 - ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita handoff. Ainda sem laudo 15-D próprio.

3. Micro-Ferramenta: `aidd-anatomy`

- **Foto / Identidade:** `aidd-anatomy`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-anatomy/SKILL.md`
- **O que Faz (Missão Única):** Extracts and documents the complete technical anatomy of an ecosystem target (pipeline, tool, or component) producing both Markdown and visual HTML artifacts in `docs/anatomias/anatomia-/. Use when dissecting a target, or when the user asks for "anatomia", "/anatomia", "desmontar ferramenta", "anatomia do alvo", "analisar anatomia", or "anatomia do pipeline".`
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-anatomy/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/anatomia`
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` cita `worktree`.
 - ▶ **D4. Componentes e Fractalidade:** scripts: `gerar_anatomia_html.py` em `scripts/`; MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Extracts and documents the complete technical anatomy of an ecosystem target (pipeline, tool, or component) producing both Markdown and visual HTML artifacts in `docs/anatomias/anatomia-/. Use when dissecting a target, or when the user asks for "anatomia", "/anatomia", "desmontar ferramenta", "anatomia do alvo", "analisar anatomia", or "anatomia do pipeline".`
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md` e os scripts locais.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.
 - ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py componentes sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita rollback nem limpeza.
 - ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` cita `handoff`. Ainda sem laudo 15-D próprio.

4. Micro-Ferramenta: `aidd-audit-4f`

- **Foto / Identidade:** `aidd-audit-4f`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-audit-4f/SKILL.md`
- **O que Faz (Missão Única):** Runs the linear 4-phase audit pipeline (Inspector, Architect, Builder, Return) on an ecosystem tool through the 15-D lens, in a cycle folder `docs/auditoria` an own branch, merging only on human approval. Use when the user wants to audit a tool, or types `"/audit-4f", "/aidd-auditor", "audit-4f", "auditoria 4F", "auditar ferramenta", "rodar pipeline de auditoria".`
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-audit-4f/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita `worktree` nem isolamento.

- ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em [SKILL.md](#)); MCPs citados: nenhum.
- ▶ **D5. Visão e Escopo:** Runs the linear 4-phase audit pipeline (Inspector, Architect, Builder, Return) on an ecosystem tool through the 15-D lens, in a cycle folder docs/auditoria own branch, merging only on human approval. Use when the user wants to audit a tool, or types "/audit-4f", "/aidd-auditor", "audit-4f", "auditoria 4F", "auditar ferramenta", "rodar pipeline de auditoria".
- ▶ **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#).
- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
- ▶ **D9. O que o Estágio Entrega:** o que o roteiro do [SKILL.md](#) orienta produzir (ver D5); não há motor próprio que grave arquivo.
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o [SKILL.md](#) cita `fallback`.
- ▶ **D12. Observabilidade e Frugalidade:** o [SKILL.md](#) cita `log`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o [SKILL.md](#) não cita rollback nem limpeza.
- ▶ **D15. Output Consolidado e Handoff:** o [SKILL.md](#) não cita handoff. Ainda sem laudo 15-D próprio.

5. Micro-Ferramenta: [aidd-components](#)

- **Foto / Identidade:** [aidd-components](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-components/SKILL.md](#)
- **O que Faz (Missão Única):** Creates or updates agnostic ecosystem components (skills, commands, hooks, mcps, specs, configs, sub-agents, scripts) in the single source componentes/ and distributes them to every harness with components sync and verify. Use when the user asks to create, update or sync a component, or says "componente", "sincronizar componentes", "components sync", "distribuir para os harnesses".
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no [SKILL.md](#); distribuída pela fonte única [componentes/compartilhado/skills/aidd-components/](#).
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o [SKILL.md](#) não cita worktree nem isolamento.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em [SKILL.md](#)); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Creates or updates agnostic ecosystem components (skills, commands, hooks, mcps, specs, configs, sub-agents, scripts) in the single source componentes/ and distributes them to every harness with components sync and verify. Use when the user asks to create, update or sync a component, or says "componente", "sincronizar componentes", "components sync", "distribuir para os harnesses".
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#).
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
 - ▶ **D9. O que o Estágio Entrega:** o que o roteiro do [SKILL.md](#) orienta produzir (ver D5); não há motor próprio que grave arquivo.

- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
- ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` não cita log, telemetria nem `secoes/`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita rollback nem limpeza.
- ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita handoff. Ainda sem laudo 15-D próprio.

6. Micro-Ferramenta: `aidd-dataviz`

- **Foto / Identidade:** `aidd-dataviz`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-dataviz/SKILL.md`
- **O que Faz (Missão Única):** Validates categorical and ordinal chart palettes with the deterministic OKLCH/OKLab script `validate_palette.py` (lightness band, chroma floor, CVD separation, normal-vision floor, WCAG contrast). Use when the user wants to check chart colors, "validate palette", "are these series colors accessible", "palette fails CVD", sequential ramp checks, or before shipping any dashboard/chart color set.
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-dataviz/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita `worktree` nem isolamento.
 - ▶ **D4. Componentes e Fractalidade:** scripts: `validate_palette.py` em `scripts/`; MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Validates categorical and ordinal chart palettes with the deterministic OKLCH/OKLab script `validate_palette.py` (lightness band, chroma floor, CVD separation, normal-vision floor, WCAG contrast). Use when the user wants to check chart colors, "validate palette", "are these series colors accessible", "palette fails CVD", sequential ramp checks, or before shipping any dashboard/chart color set.
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md` e os scripts locais.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.
 - ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` não cita log, telemetria nem `secoes/`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita rollback nem limpeza.
 - ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita handoff. Ainda sem laudo 15-D próprio.

7. Micro-Ferramenta: `aidd-delivery`

- **Foto / Identidade:** `aidd-delivery`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-delivery/SKILL.md`

- **O que Faz (Missão Única):** Provides the delivery template with before/after evidence and real exit codes, rollback door and blast radius. Use when writing a long commit body, a PR body, a RELATORIO-CONSTRUTOR.md entry, or closing a ticket, or the user says "entrega", "fechar ticket".
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-delivery/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` cita `worktree`.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Provides the delivery template with before/after evidence and real exit codes, rollback door and blast radius. Use when writing a long commit body, a PR body, a RELATORIO-CONSTRUTOR.md entry, or closing a ticket, or the user says "entrega", "fechar ticket".
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md`.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
 - ▶ **D9. O que o Estágio Entrega:** o que o roteiro do `SKILL.md` orienta produzir (ver D5); não há motor próprio que grave arquivo.
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py componentes sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` não cita log, telemetria nem `secoes/`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` cita `rollback`, `reverter`.
 - ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita handoff. Ainda sem laudo 15-D próprio.

8. Micro-Ferramenta: `aidd-dependencies`

- **Foto / Identidade:** `aidd-dependencies`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-dependencies/SKILL.md`
- **O que Faz (Missão Única):** Registers, installs and verifies third-party skills and MCP servers the agent itself uses in this repository (e.g. impeccable, Cloudflare skills, Playwright, GitHub) through `gates/dependencias_externas.json`. Use when the user wants to bootstrap after a clone, add a third-party skill or MCP, or says `"/dependencia"`, `"dependência"`, `"instalar skill de terceiro"`, `"adicionar MCP"`. Not for MCPs that ship inside a generated product (that is `aidd-mcp`).
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-dependencies/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/dependencia`
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita `worktree` nem isolamento.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Registers, installs and verifies third-party skills and MCP servers the agent itself uses in this repository (e.g. impeccable, Cloudflare skills, Playwright, GitHub) through `gates/dependencias_externas.json`. Use when the user wants to bootstrap after a

clone, add a third-party skill or MCP, or says "/dependencia", "dependência", "instalar skill de terceiro", "adicionar MCP". Not for MCPs that ship inside a generated product (that is aidd-mcp).

- ▶ **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#).
- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
- ▶ **D9. O que o Estágio Entrega:** o que o roteiro do [SKILL.md](#) orienta produzir (ver D5); não há motor próprio que grave arquivo.
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o [SKILL.md](#) cita [fallback](#).
- ▶ **D12. Observabilidade e Frugalidade:** o [SKILL.md](#) não cita log, telemetria nem [secoes/](#).
- ▶ **D13. Quality Gates (Portões):** [G_SKILL_ROT](#) confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o [SKILL.md](#) não cita rollback nem limpeza.
- ▶ **D15. Output Consolidado e Handoff:** o [SKILL.md](#) não cita handoff. Ainda sem laudo 15-D próprio.

9. Micro-Ferramenta: [aid-diagnose](#)

- **Foto / Identidade:** [aid-diagnose](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aid-diagnose/SKILL.md](#)
- **O que Faz (Missão Única):** Systematic scientific fault triage using hypothesis isolation, regression testing, and code review graph. Use when the user reports a bug, a failing test or unexpected behavior, or says "diagnosticar", "investigar bug", "por que quebrou".
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter [name/description](#) no [SKILL.md](#); distribuída pela fonte única [componentes/compartilhado/skills/aid-diagnose/](#).
 - ▶ **D2. Input e Gatilhos:** gatilho pela [description](#) do frontmatter (nenhum arquivo em [comandos/](#) aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o [SKILL.md](#) cita [worktree](#), [isolamento](#).
 - ▶ **D4. Componentes e Fractalidade:** scripts: [cli.py](#), [cobertura_grafo.py](#), [fallback.py](#), [handoff.py](#), [isolamento.py](#), [observabilidade.py](#), [rollback.py](#) em [scripts/](#); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Systematic scientific fault triage using hypothesis isolation, regression testing, and code review graph. Use when the user reports a bug, a failing test or unexpected behavior, or says "diagnosticar", "investigar bug", "por que quebrou".
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#) e os scripts locais.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.
 - ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o [SKILL.md](#) cita [fallback](#), [retry](#), [indisponível](#).
 - ▶ **D12. Observabilidade e Frugalidade:** o [SKILL.md](#) cita [log](#).
 - ▶ **D13. Quality Gates (Portões):** [G_SKILL_ROT](#) confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Portão próprio: [G_aid_diagnose.py](#).
 - ▶ **D14. Critério de Rejeição (Rollback):** o [SKILL.md](#) cita [rollback](#), [limpeza](#).

- ▶ **D15. Output Consolidado e Handoff:** o [SKILL.md](#) cita [handoff](#). Laudo 15-D: [docs/auditoria/aidd-diagnose/ciclo-01/LAUDO-15D-INICIAL.md](#), [docs/auditoria/aidd-diagnose/ciclo-01/LAUDO-15D-REVISADO.md](#).

10. Micro-Ferramenta: [aidd-dispatch](#)

- **Foto / Identidade:** [aidd-dispatch](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-dispatch/SKILL.md](#)
- **O que Faz (Missão Única):** Dispatches VSA vertical slices into ephemeral git worktrees in topological (DAG) batches, with strict file boundaries, quality gates per slice and safe convergence into the aidd-master modular monolith. Use when the user wants to execute a `PLANNER.json` or `vsa_dispatch.json` as slices, or types `"/dispatch"`, `"/aidd-dispatch"`, `"dispatch"`, `"despachar fatias"`.
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no [SKILL.md](#); distribuída pela fonte única [componentes/compartilhado/skills/aidd-dispatch/](#).
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em [comandos/](#) aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o [SKILL.md](#) cita [worktree](#), [isolamento](#).
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em [SKILL.md](#)); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Dispatches VSA vertical slices into ephemeral git worktrees in topological (DAG) batches, with strict file boundaries, quality gates per slice and safe convergence into the aidd-master modular monolith. Use when the user wants to execute a `PLANNER.json` or `vsa_dispatch.json` as slices, or types `"/dispatch"`, `"/aidd-dispatch"`, `"dispatch"`, `"despachar fatias"`.
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#).
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
 - ▶ **D9. O que o Estágio Entrega:** o que o roteiro do [SKILL.md](#) orienta produzir (ver D5); não há motor próprio que grave arquivo.
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o [SKILL.md](#) cita [fallback](#).
 - ▶ **D12. Observabilidade e Frugalidade:** o [SKILL.md](#) cita [log](#).
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o [SKILL.md](#) cita [limpeza](#).
 - ▶ **D15. Output Consolidado e Handoff:** o [SKILL.md](#) não cita [handoff](#). Ainda sem laudo 15-D próprio.

11. Micro-Ferramenta: [aidd-docx](#)

- **Foto / Identidade:** [aidd-docx](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-docx/SKILL.md](#)
- **O que Faz (Missão Única):** Use when creating, reading, editing, or manipulating Word documents (.docx) or Word templates (.dotx), including reports, memos, letters, tracked changes, formatting, or document conversion.
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no [SKILL.md](#); distribuída pela fonte única [componentes/compartilhado/skills/aidd-docx/](#).
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em [comandos/](#) aponta para ela).

- ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita worktree nem isolamento.
- ▶ **D4. Componentes e Fractalidade:** scripts: `accept_changes.py`, `comment.py`, `merge_runs.py` em `scripts/`; MCPs citados: nenhum.
- ▶ **D5. Visão e Escopo:** Use when creating, reading, editing, or manipulating Word documents (.docx) or Word templates (.dotx), including reports, memos, letters, tracked changes, formatting, or document conversion.
- ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md` e os scripts locais.
- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.
- ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`, `indisponível`.
- ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` não cita log, telemetria nem `secoes/`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita rollback nem limpeza.
- ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita handoff. Ainda sem laudo 15-D próprio.

12. Micro-Ferramenta: `aidd-enterprise`

- **Foto / Identidade:** `aidd-enterprise`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-enterprise/SKILL.md`
- **O que Faz (Missão Única):** Injects and audits mission-critical components with JSON Schema validation, SHA-256 integrity and automatic rollback. Use when the user wants to inject a certified skill, rule, mcp, spec, config, hook or agent, or says "enterprise", "/enterprise", "injetar componente", "blindagem SHA-256".
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-enterprise/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/enterprise`
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita worktree nem isolamento.
 - ▶ **D4. Componentes e Fractalidade:** scripts: `cli.py`, `fallback.py`, `injetor.py`, `isolamento.py`, `observabilidade.py`, `orquestrador.py`, `rollback.py` em `scripts/`; MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Injects and audits mission-critical components with JSON Schema validation, SHA-256 integrity and automatic rollback. Use when the user wants to inject a certified skill, rule, mcp, spec, config, hook or agent, or says "enterprise", "/enterprise", "injetar componente", "blindagem SHA-256".
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md` e os scripts locais.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.
 - ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.

- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Portão próprio: `G_aidd_enterprise.py`.
- ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` cita `rollback`.
- ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita `handoff`. Laudo 15-D: `docs/auditoria/aidd-enterprise/ciclo-01/LAUDO-15D-INICIAL.md`, `docs/auditoria/aidd-enterprise/ciclo-01/LAUDO-15D-REVISADO.md`.

13. Micro-Ferramenta: `aidd-evolution`

- **Foto / Identidade:** `aidd-evolution`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-evolution/SKILL.md`
- **O que Faz (Missão Única):** Runs the technical evolution pipeline that executes, ticket by ticket, the PLANO-EVOLUCAO of an audit cycle in ephemeral git worktrees, with a gate before every commit and merge only on human approval. Use when the user wants to evolve a tool from its evolution plan, or types `"/evolucao"`, `"/aidd-evolucao"`, `"evolucao"`, `"evoluir ferramenta"`, `"rodar pipeline de evolucao"`.
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-evolution/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/evolucao`
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` cita `worktree`, `isolamento`.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Runs the technical evolution pipeline that executes, ticket by ticket, the PLANO-EVOLUCAO of an audit cycle in ephemeral git worktrees, with a gate before every commit and merge only on human approval. Use when the user wants to evolve a tool from its evolution plan, or types `"/evolucao"`, `"/aidd-evolucao"`, `"evolucao"`, `"evoluir ferramenta"`, `"rodar pipeline de evolucao"`.
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md`.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
 - ▶ **D9. O que o Estágio Entrega:** o que o roteiro do `SKILL.md` orienta produzir (ver D5); não há motor próprio que grave arquivo.
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py componentes sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita `rollback` nem `limpeza`.
 - ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita `handoff`. Ainda sem laudo 15-D próprio.

14. Micro-Ferramenta: `aidd-forge`

- **Foto / Identidade:** `aidd-forge`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-forge/SKILL.md`
- **O que Faz (Missão Única):** Bootstraps and hardens a target repository with the aidd-forge governance kit (quality gates, git hooks, token-economy rules, phase isolation). Use when the user asks to bootstrap, initialize or harden a project, or says `"forge"`, `"/forge"`, `"blindar projeto"`, `"inicializar governanca"`.
- **Lente 15-D desta Micro-Ferramenta:**

- ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-forge/`.
- ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/forge`
- ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` cita `isolamento`.
- ▶ **D4. Componentes e Fractalidade:** scripts: `bootstrap.py`, `cli.py`, `isolamento.py`, `observabilidade.py`, `orquestracao.py`, `resiliencia.py`, `rollback.py` em `scripts/`; MCPs citados: nenhum.
- ▶ **D5. Visão e Escopo:** Bootstraps and hardens a target repository with the aidd-forge governance kit (quality gates, git hooks, token-economy rules, phase isolation). Use when the user asks to bootstrap, initialize or harden a project, or says "forge", "/forge", "blindar projeto", "inicializar governança".
- ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md` e os scripts locais.
- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.
- ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py componentes sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
- ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Portão próprio: `G_aidd_forge.py`.
- ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` cita `rollback`.
- ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` cita `handoff`. Laudo 15-D: `docs/auditoria/aidd-forge/ciclo-01/LAUDO-15D-INICIAL.md`, `docs/auditoria/aidd-forge/ciclo-01/LAUDO-15D-REVISADO.md`, `docs/auditoria/aidd-forge/ciclo-02/LAUDO-15D-INICIAL.md`, `docs/auditoria/aidd-forge/ciclo-02/LAUDO-15D-REVISADO.md`.

15. Micro-Ferramenta: `aidd-freedom`

- **Foto / Identidade:** `aidd-freedom`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-freedom/SKILL.md`
- **O que Faz (Missão Única):** Runs Triad Flow 03 (low-code liberation) end to end - removes Lovable, v0 or Bolt vendor lock-in, migrates Supabase to PostgreSQL through the aidd-freedom engine while keeping the UI, then master, enterprise and ops; also runs single engine steps (scan, convert-db, merge, pack). Use when the user wants to free or self-host a low-code app or asks for one of those steps, or types "/freedom", "freedom", "freedom-motor", "/bridge", "bridge", "libertar app lovable", "desacoplar app low-code", "migrar v0 ou bolt para postgresql", "scan do lovable", "converter banco", "empacotar para VPS".
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-freedom/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/bridge`, `/freedom`
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita `worktree` nem `isolamento`.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Runs Triad Flow 03 (low-code liberation) end to end - removes Lovable, v0 or Bolt vendor lock-in, migrates Supabase to PostgreSQL through the aidd-freedom engine while keeping the UI, then master, enterprise and ops; also runs single engine steps (scan, convert-db, merge, pack). Use when the user wants to free or self-host a low-code app or asks for one of those steps, or types "/freedom", "freedom", "freedom-motor", "/"

bridge", "bridge", "libertar app lovable", "desacoplar app low-code", "migrar v0 ou bolt para postgresql", "scan do lovable", "converter banco", "empacotar para VPS".

- ▶ **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#).
- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
- ▶ **D9. O que o Estágio Entrega:** o que o roteiro do [SKILL.md](#) orienta produzir (ver D5); não há motor próprio que grave arquivo.
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o [SKILL.md](#) cita `fallback`, `retry`.
- ▶ **D12. Observabilidade e Frugalidade:** o [SKILL.md](#) cita `log`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o [SKILL.md](#) não cita rollback nem limpeza.
- ▶ **D15. Output Consolidado e Handoff:** o [SKILL.md](#) cita `handoff`. Laudo 15-D: [docs/auditoria/aidd-freedom/ciclo-02/LAUDO-15D-INICIAL.md](#), [docs/auditoria/aidd-freedom/ciclo-02/LAUDO-15D-REVISADO.md](#).

16. Micro-Ferramenta: [aidd-frontend-forms](#)

- **Foto / Identidade:** [aidd-frontend-forms](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-frontend-forms/SKILL.md](#)
- **O que Faz (Missão Única):** Use when creating, reviewing, or refactoring typed frontend forms in Next.js using Zod validation schemas and React Hook Form.
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no [SKILL.md](#); distribuída pela fonte única [componentes/compartilhado/skills/aidd-frontend-forms/](#).
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o [SKILL.md](#) não cita worktree nem isolamento.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em [SKILL.md](#)); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Use when creating, reviewing, or refactoring typed frontend forms in Next.js using Zod validation schemas and React Hook Form.
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#).
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
 - ▶ **D9. O que o Estágio Entrega:** o que o roteiro do [SKILL.md](#) orienta produzir (ver D5); não há motor próprio que grave arquivo.
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o [SKILL.md](#) cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o [SKILL.md](#) não cita `log`, `telemetria` nem `secoes/`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o [SKILL.md](#) não cita rollback nem limpeza.
 - ▶ **D15. Output Consolidado e Handoff:** o [SKILL.md](#) não cita `handoff`. Ainda sem laudo 15-D próprio.

17. Micro-Ferramenta: [aidd-grill](#)

- **Foto / Identidade:** [aidd-grill](#)

- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-grill/SKILL.md`
- **O que Faz (Missão Única):** Socratic interview that resolves assumptions, trade-offs and invariants before code. Use when requirements are open, or "grill".
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-grill/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita `worktree` nem `isolamento`.
 - ▶ **D4. Componentes e Fractalidade:** scripts: `cli.py`, `fallback.py`, `handoff.py`, `isolamento.py`, `motor.py`, `observabilidade.py`, `parser.py`, `rollback.py` em `scripts/`; MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Socratic interview that resolves assumptions, trade-offs and invariants before code. Use when requirements are open, or "grill".
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md` e os scripts locais.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.
 - ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py componentes sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Portão próprio: `G_aidd_grill.py`.
 - ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita `rollback` nem `limpeza`.
 - ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita `handoff`. Laudo 15-D: `docs/auditoria/aidd-grill/ciclo-01/LAUDO-15D-INICIAL.md`, `docs/auditoria/aidd-grill/ciclo-01/LAUDO-15D-REVISADO.md`.

18. Micro-Ferramenta: `aidd-grill-docs`

- **Foto / Identidade:** `aidd-grill-docs`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-grill-docs/SKILL.md`
- **O que Faz (Missão Única):** Socratic interview grounded in existing repository architecture, domain documentation, and invariant rules. Use when a change must be checked against `MEMORY.md`, `AGENTS.md` laws or domain docs before coding, or the user says "grill com docs", "confrontar com a arquitetura".
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-grill-docs/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` cita `isolamento`.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Socratic interview grounded in existing repository architecture, domain documentation, and invariant rules. Use when a change must be checked against `MEMORY.md`, `AGENTS.md` laws or domain docs before coding, or the user says "grill com docs", "confrontar com a arquitetura".
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md`.

- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
- ▶ **D9. O que o Estágio Entrega:** o que o roteiro do `SKILL.md` orienta produzir (ver D5); não há motor próprio que grave arquivo.
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
- ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita rollback nem limpeza.
- ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` cita `handoff`. Ainda sem laudo 15-D próprio.

19. Micro-Ferramenta: `aidd-handoff`

- **Foto / Identidade:** `aidd-handoff`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-handoff/SKILL.md`
- **O que Faz (Missão Única):** Serializes and compacts session state into a structured markdown artifact for context rotation or agent handover. Use when the context is heavy, the session ends or work passes to another agent, or the user says "handoff", "passar o bastão", "salvar contexto".
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-handoff/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` cita `worktree`.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Serializes and compacts session state into a structured markdown artifact for context rotation or agent handover. Use when the context is heavy, the session ends or work passes to another agent, or the user says "handoff", "passar o bastão", "salvar contexto".
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md`.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
 - ▶ **D9. O que o Estágio Entrega:** o que o roteiro do `SKILL.md` orienta produzir (ver D5); não há motor próprio que grave arquivo.
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`, `secoes/`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita rollback nem limpeza.
 - ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` cita `handoff`. Ainda sem laudo 15-D próprio.

20. Micro-Ferramenta: `aidd-improvement`

- **Foto / Identidade:** `aidd-improvement`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-improvement/SKILL.md`
- **O que Faz (Missão Única):** Deeply investigates a requested ecosystem improvement in the real code and writes a scored evaluation report (current grade 0-10 with evidence) in docs/

melhorias/. Step 1 of the melhoria → plan → orchestrate flow. Use when the user asks to analyze, evaluate or grade an improvement or refactor before planning, or to re-analyze an existing plan, or types "/melhoria", "melhoria", "analisar melhoria", "avaliar melhoria", "reanalisar plano".

- **Lente 15-D desta Micro-Ferramenta:**

- ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-improvement/`.
- ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/melhoria`
- ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` cita `isolamento`.
- ▶ **D4. Componentes e Fractalidade:** scripts: `analizador.py`, `cli.py`, `fallback.py`, `handoff.py`, `isolamento.py`, `observabilidade.py`, `rollback.py` em `scripts/`; MCPs citados: nenhum.
- ▶ **D5. Visão e Escopo:** Deeply investigates a requested ecosystem improvement in the real code and writes a scored evaluation report (current grade 0-10 with evidence) in `docs/melhorias/`. Step 1 of the melhoria → plan → orchestrate flow. Use when the user asks to analyze, evaluate or grade an improvement or refactor before planning, or to re-analyze an existing plan, or types "/melhoria", "melhoria", "analisar melhoria", "avaliar melhoria", "reanalisar plano".
- ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md` e os scripts locais.
- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.
- ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py componentes sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`, `indisponível`.
- ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` cita `rollback`.
- ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` cita `handoff`. Ainda sem laudo 15-D próprio.

21. Micro-Ferramenta: `aidd-ingest`

- **Foto / Identidade:** `aidd-ingest`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-ingest/SKILL.md`
- **O que Faz (Missão Única):** Ingests external repositories (tools, skills, MCPs, processes), triages adherence against the 13 Inviolable Laws, runs the 4F audit pipeline, generates an evolution plan, and executes migration tickets in ephemeral worktrees. Use when the user wants to import, ingest, or adopt an external Git repository or tool into the ecosystem, or types "/ingest", "ingest", "ingerir repositório", "adotar ferramenta externa".
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-ingest/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` cita `worktree`, `isolamento`.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Ingests external repositories (tools, skills, MCPs, processes), triages adherence against the 13 Inviolable Laws, runs the 4F audit pipeline, generates an evolution plan, and executes migration tickets in ephemeral worktrees. Use when the user wants to

import, ingest, or adopt an external Git repository or tool into the ecosystem, or types "/ ingest", "ingest", "ingerir repositório", "adotar ferramenta externa".

- ▶ **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#).
- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
- ▶ **D9. O que o Estágio Entrega:** o que o roteiro do [SKILL.md](#) orienta produzir (ver D5); não há motor próprio que grave arquivo.
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o [SKILL.md](#) cita `fallback`.
- ▶ **D12. Observabilidade e Frugalidade:** o [SKILL.md](#) cita `log`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o [SKILL.md](#) não cita rollback nem limpeza.
- ▶ **D15. Output Consolidado e Handoff:** o [SKILL.md](#) não cita handoff. Ainda sem laudo 15-D próprio.

22. Micro-Ferramenta: [aidd-master](#)

- **Foto / Identidade:** [aidd-master](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-master/SKILL.md](#)
- **O que Faz (Missão Única):** Scaffolds and integrates self-contained vertical slices in the aidd-master modular monolith (models, services, REST routes, contract tests, UI). Use when the user wants a new module or slice, or says "master", "/master", "nova fatia vertical", "adicionar módulo".
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no [SKILL.md](#); distribuída pela fonte única [componentes/compartilhado/skills/aidd-master/](#).
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/master`
 - ▶ **D3. Raio de Impacto e Isolamento:** o [SKILL.md](#) não cita worktree nem isolamento.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em [SKILL.md](#)); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Scaffolds and integrates self-contained vertical slices in the aidd-master modular monolith (models, services, REST routes, contract tests, UI). Use when the user wants a new module or slice, or says "master", "/master", "nova fatia vertical", "adicionar módulo".
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#).
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
 - ▶ **D9. O que o Estágio Entrega:** o que o roteiro do [SKILL.md](#) orienta produzir (ver D5); não há motor próprio que grave arquivo.
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o [SKILL.md](#) cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o [SKILL.md](#) cita `log`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o [SKILL.md](#) não cita rollback nem limpeza.
 - ▶ **D15. Output Consolidado e Handoff:** o [SKILL.md](#) não cita handoff. Ainda sem laudo 15-D próprio.

23. Micro-Ferramenta: `aidd-mcp`

- **Foto / Identidade:** `aidd-mcp`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-mcp/SKILL.md`
- **O que Faz (Missão Única):** Builds a new Model Context Protocol (MCP) server that ships inside an ecosystem tool or generated product, with its single source in `componentes/components/sync`. Use when the user wants to create or change an own MCP server, or says "criar MCP", "novo servidor MCP", "expor ferramenta via MCP". Not for third-party MCPs the agent consumes (that is `aidd-dependencies`).
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-mcp/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita `worktree` nem isolamento.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: `code-review-graph`.
 - ▶ **D5. Visão e Escopo:** Builds a new Model Context Protocol (MCP) server that ships inside an ecosystem tool or generated product, with its single source in `componentes/distribution` through `components/sync`. Use when the user wants to create or change an own MCP server, or says "criar MCP", "novo servidor MCP", "expor ferramenta via MCP". Not for third-party MCPs the agent consumes (that is `aidd-dependencies`).
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md`.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
 - ▶ **D9. O que o Estágio Entrega:** o que o roteiro do `SKILL.md` orienta produzir (ver D5); não há motor próprio que grave arquivo.
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py componentes sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` não cita log, telemetria nem `secoes/`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita rollback nem limpeza.
 - ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita handoff. Ainda sem laudo 15-D próprio.

24. Micro-Ferramenta: `aidd-open`

- **Foto / Identidade:** `aidd-open`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-open/SKILL.md`
- **O que Faz (Missão Única):** Runs Triad Flow 02 (open-source engines) end to end - curates tested open-source engines, integrates them as VSA slices through the `aidd-open` engine, then master, enterprise and ops; also runs only the engine that generates a multi-service stack from a `PLANO-INFRAESTRUTURA.json`. Use when the user wants an app built on open-source engines or a deployable stack from an infrastructure plan, or types `"/open"`, `"/aidd-open"`, `"open"`, `"open-motor"`, `"/factory"`, `"factory"`, `"criar com open-source"`, `"integrar motor open source"`, `"gerar stack"`, `"gerar gateway"`, `"gerar compose"`.
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-open/`.

- ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/factory`, `/open`
- ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` cita `isolamento`.
- ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.
- ▶ **D5. Visão e Escopo:** Runs Triad Flow 02 (open-source engines) end to end - curates tested open-source engines, integrates them as VSA slices through the aidd-open engine, then master, enterprise and ops; also runs only the engine that generates a multi-service stack from a PLANO-INFRAESTRUTURA.json. Use when the user wants an app built on open-source engines or a deployable stack from an infrastructure plan, or types `"/open"`, `"/aidd-open"`, `"open"`, `"open-motor"`, `"/factory"`, `"factory"`, `"criar com open-source"`, `"integrar motor open source"`, `"gerar stack"`, `"gerar gateway"`, `"gerar compose"`.
- ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md`.
- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
- ▶ **D9. O que o Estágio Entrega:** o que o roteiro do `SKILL.md` orienta produzir (ver D5); não há motor próprio que grave arquivo.
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
- ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita rollback nem limpeza.
- ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` cita `handoff`. Ainda sem laudo 15-D próprio.

25. Micro-Ferramenta: `aidd-ops`

- **Foto / Identidade:** `aidd-ops`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-ops/SKILL.md`
- **O que Faz (Missão Única):** Runs the aidd-ops agentic infrastructure pipeline (VPS sizing, SSH hardening, Docker, Cloudflare, deploy). Use when the user wants to provision or deploy infrastructure, or says `"ops"`, `"/ops"`, `"infraestrutura"`, `"subir na VPS"`, `"provisionar"`.
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-ops/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/ops`
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita `worktree` nem `isolamento`.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Runs the aidd-ops agentic infrastructure pipeline (VPS sizing, SSH hardening, Docker, Cloudflare, deploy). Use when the user wants to provision or deploy infrastructure, or says `"ops"`, `"/ops"`, `"infraestrutura"`, `"subir na VPS"`, `"provisionar"`.
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md`.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
 - ▶ **D9. O que o Estágio Entrega:** o que o roteiro do `SKILL.md` orienta produzir (ver D5); não há motor próprio que grave arquivo.
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.

- ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita rollback nem limpeza.
- ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` cita `handoff`. Ainda sem laudo 15-D próprio.

26. Micro-Ferramenta: `aidd-orca`

- **Foto / Identidade:** `aidd-orca`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-orca/SKILL.md`
- **O que Faz (Missão Única):** Native git-worktree engine that executes multi-phase plans (00-PROCESSO-E-DECSOES.md + NN-*.md) with ephemeral worktrees, per-front harness choice, local gate audit before merge and an anti-loop circuit breaker. Called by `aidd-orchestrate` when the user picks the native git worktree environment. Use when that environment was chosen, or the user says "worktree nativo", "motor de worktree", "orca nativo".
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-orca/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/orchestrate`
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` cita `worktree`.
 - ▶ **D4. Componentes e Fractalidade:** scripts: `agent_spawner.py`, `circuit_breaker.py`, `flight_plan.py`, `gate_auditor.py`, `hooks.py`, `orca_real_plan.py`, `orchestrator_engine.py`, `plan_io.py`, `plan_parser.py`, `state_engine.py`, `subagent_plan.py`, `worktree_engine.py` em `scripts/`; MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Native git-worktree engine that executes multi-phase plans (00-PROCESSO-E-DECSOES.md + NN-*.md) with ephemeral worktrees, per-front harness choice, local gate audit before merge and an anti-loop circuit breaker. Called by `aidd-orchestrate` when the user picks the native git worktree environment. Use when that environment was chosen, or the user says "worktree nativo", "motor de worktree", "orca nativo".
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md` e os scripts locais.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.
 - ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py componentes sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`, `retry`.
 - ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita rollback nem limpeza.
 - ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita `handoff`. Ainda sem laudo 15-D próprio.

27. Micro-Ferramenta: `aidd-orchestrate`

- **Foto / Identidade:** `aidd-orchestrate`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-orchestrate/SKILL.md`
- **O que Faz (Missão Única):** Routes the execution of an approved plan to one of three environments (ORCA app via `orca-cli`, subagents of this session, or native git worktrees), compiles the per-environment Flight Plan and runs it only after explicit approval. Step 3 of the melhoria → plan → orchestrate flow. Use when the user wants to execute an approved plan, or types `/orchestrate`, `orchestrate`, `executar plano`, `orquestrar plano`, `plano de voo`.

- **Lente 15-D desta Micro-Ferramenta:**

- ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-orchestrate/`.
- ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/orchestrate`
- ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` cita `worktree`, `isolamento`.
- ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.
- ▶ **D5. Visão e Escopo:** Routes the execution of an approved plan to one of three environments (ORCA app via `orca-cli`, subagents of this session, or native git worktrees), compiles the per-environment Flight Plan and runs it only after explicit approval. Step 3 of the melhoria → plan → orchestrate flow. Use when the user wants to execute an approved plan, or types `"/orchestrate"`, `"orchestrate"`, `"executar plano"`, `"orquestrar plano"`, `"plano de voo"`.
- ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md`.
- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
- ▶ **D9. O que o Estágio Entrega:** o que o roteiro do `SKILL.md` orienta produzir (ver D5); não há motor próprio que grave arquivo.
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py componentes sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`, `retry`.
- ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` cita `limpeza`.
- ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita handoff. Ainda sem laudo 15-D próprio.

28. Micro-Ferramenta: `aidd-pdf`

- **Foto / Identidade:** `aidd-pdf`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-pdf/SKILL.md`
- **O que Faz (Missão Única):** Use when processing PDF files, including reading or extracting text/tables, combining or splitting PDFs, rotating pages, watermarking, form filling, OCR on scanned pages, or generating new PDFs.
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-pdf/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita `worktree` nem `isolamento`.
 - ▶ **D4. Componentes e Fractalidade:** scripts: `check_bounding_boxes.py`, `check_fillable_fields.py`, `convert_pdf_to_images.py`, `create_validation_image.py`, `extract_form_field_info.py`, `extract_form_structure.py`, `fill_fillable_fields.py`, `fill_pdf_form_with_annotations.py` em `scripts/`; MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Use when processing PDF files, including reading or extracting text/tables, combining or splitting PDFs, rotating pages, watermarking, form filling, OCR on scanned pages, or generating new PDFs.
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md` e os scripts locais.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.

- ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
- ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` não cita log, telemetria nem `secoes/`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita rollback nem limpeza.
- ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita handoff. Ainda sem laudo 15-D próprio.

29. Micro-Ferramenta: `aidd-pipeline`

- **Foto / Identidade:** `aidd-pipeline`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-pipeline/SKILL.md`
- **O que Faz (Missão Única):** Runs the deterministic Triad pipeline in ephemeral git worktrees with a join barrier, from a Markdown plan (run-plan) or a JSON handoff manifest (pipeline). Use when the user wants to execute plan tickets in parallel worktrees, or types `"/run-plan"`, `"/pipeline"`, `"run-plan"`, `"pipeline"`, "rodar plano em worktrees".
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-pipeline/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` cita `worktree`.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Runs the deterministic Triad pipeline in ephemeral git worktrees with a join barrier, from a Markdown plan (run-plan) or a JSON handoff manifest (pipeline). Use when the user wants to execute plan tickets in parallel worktrees, or types `"/run-plan"`, `"/pipeline"`, `"run-plan"`, `"pipeline"`, "rodar plano em worktrees".
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md`.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
 - ▶ **D9. O que o Estágio Entrega:** o que o roteiro do `SKILL.md` orienta produzir (ver D5); não há motor próprio que grave arquivo.
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` cita `limpeza`.
 - ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` cita `handoff`. Ainda sem laudo 15-D próprio.

30. Micro-Ferramenta: `aidd-plan`

- **Foto / Identidade:** `aidd-plan`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-plan/SKILL.md`
- **O que Faz (Missão Única):** Creates and validates ecosystem improvement plan folders in docs/ planos/ (PLAN-NNNN-, items in draft with current/target grades and evidence), then approves

them only on explicit user approval. Step 2 of the melhoria → plan → orchestrate flow. Use when the user wants to turn a report or request into a plan, or types `"/plan"`, `"plan"`, `"criar plano"`, `"gerar plano"`, `"aprovar plano"`. Not for the app blueprint PLANNER.json (that is aidd-planner).

- **Lente 15-D desta Micro-Ferramenta:**

- ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-plan/`.
- ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/plan`, `/planner`
- ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` cita `isolamento`.
- ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.
- ▶ **D5. Visão e Escopo:** Creates and validates ecosystem improvement plan folders in docs/planos/ (PLAN-NNNN-, items in draft with current/target grades and evidence), then approves them only on explicit user approval. Step 2 of the melhoria → plan → orchestrate flow. Use when the user wants to turn a report or request into a plan, or types `"/plan"`, `"plan"`, `"criar plano"`, `"gerar plano"`, `"aprovar plano"`. Not for the app blueprint PLANNER.json (that is aidd-planner).
- ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md`.
- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
- ▶ **D9. O que o Estágio Entrega:** o que o roteiro do `SKILL.md` orienta produzir (ver D5); não há motor próprio que grave arquivo.
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
- ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita rollback nem limpeza.
- ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita handoff. Ainda sem laudo 15-D próprio.

31. Micro-Ferramenta: aidd-planner

- **Foto / Identidade:** `aidd-planner`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-planner/SKILL.md`
- **O que Faz (Missão Única):** Generates and validates the canonical SDD/BDD app blueprint (PLANNER.json) that fuels the Triad flows (Generator, Factory, Bridge). Use when the user wants architecture intake, a project blueprint or a spec for a new app, or says `"planner"`, `"pré-plano"`, `"intake"`, `"blueprint"`, `"SDD"`, `"BDD"`. Not for ecosystem improvement plans in docs/planos/ (that is aidd-plan).
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-planner/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/planner`
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita worktree nem isolamento.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Generates and validates the canonical SDD/BDD app blueprint (PLANNER.json) that fuels the Triad flows (Generator, Factory, Bridge). Use when the user wants architecture intake, a project blueprint or a spec for a new app, or says `"planner"`, `"pré-`

-plano", "intake", "blueprint", "SDD", "BDD". Not for ecosystem improvement plans in docs/planos/ (that is aidd-plan).

- ▶ **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#).
- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
- ▶ **D9. O que o Estágio Entrega:** o que o roteiro do [SKILL.md](#) orienta produzir (ver D5); não há motor próprio que grave arquivo.
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o [SKILL.md](#) cita `fallback`.
- ▶ **D12. Observabilidade e Frugalidade:** o [SKILL.md](#) cita `log`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o [SKILL.md](#) não cita rollback nem limpeza.
- ▶ **D15. Output Consolidado e Handoff:** o [SKILL.md](#) cita `handoff`. Ainda sem laudo 15-D próprio.

32. Micro-Ferramenta: [aidd-pptx](#)

- **Foto / Identidade:** [aidd-pptx](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-pptx/SKILL.md](#)
- **O que Faz (Missão Única):** Use when creating, reading, editing, or manipulating PowerPoint presentations (.pptx) or templates (.potx), including slide decks, pitch decks, layouts, speaker notes, and visual QA.
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no [SKILL.md](#); distribuída pela fonte única [componentes/compartilhado/skills/aidd-pptx/](#).
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em [comandos/](#) aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o [SKILL.md](#) não cita worktree nem isolamento.
 - ▶ **D4. Componentes e Fractalidade:** scripts: `add_slide.py`, `clean.py`, `thumbnail.py` em [scripts/](#); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Use when creating, reading, editing, or manipulating PowerPoint presentations (.pptx) or templates (.potx), including slide decks, pitch decks, layouts, speaker notes, and visual QA.
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#) e os scripts locais.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.
 - ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o [SKILL.md](#) cita `fallback`, `indisponível`.
 - ▶ **D12. Observabilidade e Frugalidade:** o [SKILL.md](#) não cita `log`, telemetria nem [secoes/](#).
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o [SKILL.md](#) não cita rollback nem limpeza.
 - ▶ **D15. Output Consolidado e Handoff:** o [SKILL.md](#) não cita handoff. Ainda sem laudo 15-D próprio.

33. Micro-Ferramenta: [aidd-pure](#)

- **Foto / Identidade:** [aidd-pure](#)

- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-pure/SKILL.md](#)
- **O que Faz (Missão Única):** Runs Triad Flow 01 (build from scratch) end to end - forge, planner, the aidd-pure engine with strict TDD Red-Green, then master, enterprise and ops; also runs only the 8-phase engine that turns an idea into tested software. Use when the user wants a brand-new project from zero or an app from an idea, or types `"/pure"`, `"pure"`, `"pure-motor"`, `"/generate"`, `"generate"`, `"criar projeto do zero"`, `"projeto do zero puro"`, `"novo projeto com TDD"`, `"gerar app a partir da ideia"`.
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no [SKILL.md](#); distribuída pela fonte única [componentes/compartilhado/skills/aidd-pure/](#).
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/generate`, `/pure`
 - ▶ **D3. Raio de Impacto e Isolamento:** o [SKILL.md](#) cita `isolamento`.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em [SKILL.md](#)); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Runs Triad Flow 01 (build from scratch) end to end - forge, planner, the aidd-pure engine with strict TDD Red-Green, then master, enterprise and ops; also runs only the 8-phase engine that turns an idea into tested software. Use when the user wants a brand-new project from zero or an app from an idea, or types `"/pure"`, `"pure"`, `"pure-motor"`, `"/generate"`, `"generate"`, `"criar projeto do zero"`, `"projeto do zero puro"`, `"novo projeto com TDD"`, `"gerar app a partir da ideia"`.
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#).
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
 - ▶ **D9. O que o Estágio Entrega:** o que o roteiro do [SKILL.md](#) orienta produzir (ver D5); não há motor próprio que grave arquivo.
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o [SKILL.md](#) cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o [SKILL.md](#) cita `log`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o [SKILL.md](#) não cita rollback nem limpeza.
 - ▶ **D15. Output Consolidado e Handoff:** o [SKILL.md](#) cita `handoff`. Ainda sem laudo 15-D próprio.

34. Micro-Ferramenta: [aidd-reexplain](#)

- **Foto / Identidade:** [aidd-reexplain](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-reexplain/SKILL.md](#)
- **O que Faz (Missão Única):** Re-explains the last message in plain PT-BR using the `CONTEXT.md` glossary. Use when the user says they did not understand, `"não entendi"`, `"explica de novo"`, or `"reexplica"`.
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no [SKILL.md](#); distribuída pela fonte única [componentes/compartilhado/skills/aidd-reexplain/](#).
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o [SKILL.md](#) não cita `worktree` nem `isolamento`.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em [SKILL.md](#)); MCPs citados: nenhum.

- ▶ **D5. Visão e Escopo:** Re-explains the last message in plain PT-BR using the `CONTEXT.md` glossary. Use when the user says they did not understand, "não entendi", "explica de novo", or "reexplica".
- ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md`.
- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
- ▶ **D9. O que o Estágio Entrega:** o que o roteiro do `SKILL.md` orienta produzir (ver D5); não há motor próprio que grave arquivo.
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
- ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` não cita log, telemetria nem `secoes/`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita rollback nem limpeza.
- ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita handoff. Ainda sem laudo 15-D próprio.

35. Micro-Ferramenta: `aidd-retro`

- **Foto / Identidade:** `aidd-retro`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-retro/SKILL.md`
- **O que Faz (Missão Única):** Retrospective on a coding session. Turns each agent mistake into a proposed gate (mechanical) or review rule (judgement). Use when the user asks for a retro or post-mortem of a session.
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-retro/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita worktree nem isolamento.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Retrospective on a coding session. Turns each agent mistake into a proposed gate (mechanical) or review rule (judgement). Use when the user asks for a retro or post-mortem of a session.
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md`.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
 - ▶ **D9. O que o Estágio Entrega:** o que o roteiro do `SKILL.md` orienta produzir (ver D5); não há motor próprio que grave arquivo.
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`, `retry`.
 - ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita rollback nem limpeza.
 - ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita handoff. Ainda sem laudo 15-D próprio.

36. Micro-Ferramenta: `aidd-session`

- **Foto / Identidade:** `aidd-session`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-session/SKILL.md`
- **O que Faz (Missão Única):** Records the current agent session ID and metadata deterministically in `secoes/historico_sesoes.json` and `secoes/INDICE-SESOES.md` for traceability and context recovery. Use when the user wants to save or see the session ID, or types `"/sessao"`, `"/session"`, `"/id"`, `"/id-sessao"`, `"/salvar-id"`, `"salve o id desta sessão"`, `"qual o id desta conversa"`.
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-session/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/sessao`
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita `worktree` nem isolamento.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Records the current agent session ID and metadata deterministically in `secoes/historico_sesoes.json` and `secoes/INDICE-SESOES.md` for traceability and context recovery. Use when the user wants to save or see the session ID, or types `"/sessao"`, `"/session"`, `"/id"`, `"/id-sessao"`, `"/salvar-id"`, `"salve o id desta sessão"`, `"qual o id desta conversa"`.
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md`.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
 - ▶ **D9. O que o Estágio Entrega:** o que o roteiro do `SKILL.md` orienta produzir (ver D5); não há motor próprio que grave arquivo.
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py componentes sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `secoes/`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita `rollback` nem limpeza.
 - ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita `handoff`. Ainda sem laudo 15-D próprio.

37. Micro-Ferramenta: `aidd-skills`

- **Foto / Identidade:** `aidd-skills`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-skills/SKILL.md`
- **O que Faz (Missão Única):** Creates, improves and evaluates own agent skills following docs/protocolos/CONVENCAO-AUTORIA-SKILLS.md (`aidd-` name, "Use when" description, compact English body, single source in `componentes/`) and distributes them to every harness. Use when the user wants to create, edit, rename or review a skill, or says `"criar skill"`, `"nova skill"`, `"melhorar skill"`, `"skill-creator"`.
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-skills/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita `worktree` nem isolamento.
 - ▶ **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em `SKILL.md`); MCPs citados: nenhum.

- ▶ **D5. Visão e Escopo:** Creates, improves and evaluates own agent skills following docs/protocolos/CONVENCAO-AUTORIA-SKILLS.md (aidd- name, "Use when" description, compact English body, single source in componentes/) and distributes them to every harness. Use when the user wants to create, edit, rename or review a skill, or says "criar skill", "nova skill", "melhorar skill", "skill-creator".
- ▶ **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#).
- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
- ▶ **D9. O que o Estágio Entrega:** o que o roteiro do [SKILL.md](#) orienta produzir (ver D5); não há motor próprio que grave arquivo.
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py componentes sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o [SKILL.md](#) cita [fallback](#).
- ▶ **D12. Observabilidade e Frugalidade:** o [SKILL.md](#) não cita log, telemetria nem [secoes/](#).
- ▶ **D13. Quality Gates (Portões):** [G_SKILL_ROT](#) confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o [SKILL.md](#) não cita rollback nem limpeza.
- ▶ **D15. Output Consolidado e Handoff:** o [SKILL.md](#) não cita handoff. Laudo 15-D: [docs/auditoria/aidd-skills/ciclo-01/LAUDO-15D-INICIAL.md](#), [docs/auditoria/aidd-skills/ciclo-01/LAUDO-15D-REVISADO.md](#).

38. Micro-Ferramenta: [aidd-spec](#)

- **Foto / Identidade:** [aidd-spec](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-spec/SKILL.md](#)
- **O que Faz (Missão Única):** Turns discussions and decisions into an executable spec with binary acceptance criteria. Use when writing a spec, or "especificação".
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter [name/description](#) no [SKILL.md](#); distribuída pela fonte única [componentes/compartilhado/skills/aidd-spec/](#).
 - ▶ **D2. Input e Gatilhos:** gatilho pela [description](#) do frontmatter (nenhum arquivo em [comandos/](#) aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o [SKILL.md](#) não cita worktree nem isolamento.
 - ▶ **D4. Componentes e Fractalidade:** scripts: [cli.py](#), [fallback.py](#), [handoff.py](#), [isolamento.py](#), [motor.py](#), [observabilidade.py](#), [parser.py](#), [rollback.py](#) em [scripts/](#); MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Turns discussions and decisions into an executable spec with binary acceptance criteria. Use when writing a spec, or "especificação".
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#) e os scripts locais.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.
 - ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py componentes sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o [SKILL.md](#) cita [fallback](#).
 - ▶ **D12. Observabilidade e Frugalidade:** o [SKILL.md](#) cita [log](#).
 - ▶ **D13. Quality Gates (Portões):** [G_SKILL_ROT](#) confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Portão próprio: [G_aidd_spec.py](#).
 - ▶ **D14. Critério de Rejeição (Rollback):** o [SKILL.md](#) não cita rollback nem limpeza.

- ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` cita `handoff`. Laudo 15-D: `docs/auditoria/aidd-spec/ciclo-01/LAUDO-15D-INICIAL.md`, `docs/auditoria/aidd-spec/ciclo-01/LAUDO-15D-REVISADO.md`.

39. Micro-Ferramenta: `aidd-tdd`

- **Foto / Identidade:** `aidd-tdd`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-tdd/SKILL.md`
- **O que Faz (Missão Única):** Strict TDD with agreed seams, Red-Green loop, refactor at review, zero stubs. Use when implementing test first, or "tdd".
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-tdd/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita `worktree` nem `isolamento`.
 - ▶ **D4. Componentes e Fractalidade:** scripts: `cli.py`, `fallback.py`, `handoff.py`, `isolamento.py`, `motor_tdd.py`, `observabilidade.py`, `rollback.py`, `validador_seams.py` em `scripts/`; MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Strict TDD with agreed seams, Red-Green loop, refactor at review, zero stubs. Use when implementing test first, or "tdd".
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md` e os scripts locais.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.
 - ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py componentes sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Portão próprio: `G_aidd_tdd.py`.
 - ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` cita `limpeza`.
 - ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita `handoff`. Laudo 15-D: `docs/auditoria/aidd-tdd/ciclo-01/LAUDO-15D-INICIAL.md`, `docs/auditoria/aidd-tdd/ciclo-01/LAUDO-15D-REVISADO.md`.

40. Micro-Ferramenta: `aidd-textbook`

- **Foto / Identidade:** `aidd-textbook`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-textbook/SKILL.md`
- **O que Faz (Missão Única):** Generates and updates auditable corporate textbooks as PDF (pandoc + typst) with a macro-meso-micro structure, per-chapter traceability and a deterministic audit. Works in any repository. Use when the user wants to write or update a textbook, manual or book in PDF, or types `"/aidd-livro-texto"`, `"livro-texto"`, `"gerar livro"`, `"atualizar livro"`.
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-textbook/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` e pelos comandos `/aidd-livro-texto`
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita `worktree` nem `isolamento`.
 - ▶ **D4. Componentes e Fractalidade:** scripts: `livro.py` em `scripts/`; MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Generates and updates auditable corporate textbooks as PDF (pandoc + typst) with a macro-meso-micro structure, per-chapter traceability and a deterministic

audit. Works in any repository. Use when the user wants to write or update a textbook, manual or book in PDF, or types "/aidd-livro-texto", "livro-texto", "gerar livro", "atualizar livro".

- ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md` e os scripts locais.
- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.
- ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
- ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` não cita rollback nem limpeza.
- ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita handoff. Ainda sem laudo 15-D próprio.

41. Micro-Ferramenta: `aidd-tickets`

- **Foto / Identidade:** `aidd-tickets`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-tickets/SKILL.md`
- **O que Faz (Missão Única):** Splits a spec into vertical-slice tracer-bullet tickets with Blocked by. Use when breaking work into tasks, or "tickets".
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no `SKILL.md`; distribuída pela fonte única `componentes/compartilhado/skills/aidd-tickets/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o `SKILL.md` não cita worktree nem isolamento.
 - ▶ **D4. Componentes e Fractalidade:** scripts: `cli.py`, `fallback.py`, `grafo_dag.py`, `handoff.py`, `isolamento.py`, `observabilidade.py`, `parser.py`, `rollback.py` em `scripts/`; MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Splits a spec into vertical-slice tracer-bullet tickets with Blocked by. Use when breaking work into tasks, or "tickets".
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do `SKILL.md` e os scripts locais.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.
 - ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o `SKILL.md` cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o `SKILL.md` cita `log`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Portão próprio: `G_aidd_tickets.py`.
 - ▶ **D14. Critério de Rejeição (Rollback):** o `SKILL.md` cita `limpeza`.
 - ▶ **D15. Output Consolidado e Handoff:** o `SKILL.md` não cita handoff. Laudo 15-D: `docs/auditoria/aidd-tickets/ciclo-01/LAUDO-15D-INICIAL.md`, `docs/auditoria/aidd-tickets/ciclo-01/LAUDO-15D-REVISADO.md`.

42. Micro-Ferramenta: `aidd-visual-maps`

- **Foto / Identidade:** `aidd-visual-maps`

- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-visual-maps/SKILL.md](#)
- **O que Faz (Missão Única):** Builds or updates a visual map of one ecosystem piece type (guards, skills...) in docs/mapas-visuais/ from a PT-BR molde plus data read from the parts catalog, with tests that prove the generator bites. Use when the user wants a new map, to refresh a stale map, or says "mapa visual", "novo mapa", "mapa das skills", "mapa dos guardas", "atualizar mapa".
- **Lente 15-D desta Micro-Ferramenta:**
 - **D1. Contratos e Regras:** frontmatter [name/description](#) no [SKILL.md](#); distribuída pela fonte única [componentes/compartilhado/skills/aidd-visual-maps/](#).
 - **D2. Input e Gatilhos:** gatilho pela [description](#) do frontmatter (nenhum arquivo em [comandos/](#) aponta para ela).
 - **D3. Raio de Impacto e Isolamento:** o [SKILL.md](#) não cita worktree nem isolamento.
 - **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em [SKILL.md](#)); MCPs citados: nenhum.
 - **D5. Visão e Escopo:** Builds or updates a visual map of one ecosystem piece type (guards, skills...) in docs/mapas-visuais/ from a PT-BR molde plus data read from the parts catalog, with tests that prove the generator bites. Use when the user wants a new map, to refresh a stale map, or says "mapa visual", "novo mapa", "mapa das skills", "mapa dos guardas", "atualizar mapa".
 - **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#).
 - **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
 - **D9. O que o Estágio Entrega:** o que o roteiro do [SKILL.md](#) orienta produzir (ver D5); não há motor próprio que grave arquivo.
 - **D10. Orquestração e Topologia:** espelhada nos harnesses por [python ecossistema.py componentes sync --tipo todos](#).
 - **D11. Tratamento de Exceções e Fallback:** o [SKILL.md](#) cita [fallback](#).
 - **D12. Observabilidade e Frugalidade:** o [SKILL.md](#) não cita log, telemetria nem [secoes/](#).
 - **D13. Quality Gates (Portões):** [G_SKILL_ROT](#) confere caminhos e comandos citados. Roda automaticamente no commit via [.pre-commit-config.yaml](#). Sem portão próprio.
 - **D14. Critério de Rejeição (Rollback):** o [SKILL.md](#) não cita rollback nem limpeza.
 - **D15. Output Consolidado e Handoff:** o [SKILL.md](#) não cita handoff. Ainda sem laudo 15-D próprio.

43. Micro-Ferramenta: [aidd-wizard](#)

- **Foto / Identidade:** [aidd-wizard](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-wizard/SKILL.md](#)
- **O que Faz (Missão Única):** Generate an interactive bash wizard that walks a human through steps only they can perform. Use when provisioning infrastructure (VPS, Cloudflare), setting up credentials or CI secrets, walking a third-party dashboard, or running a one-off migration or cutover. Use only for steps the agent cannot perform itself.
- **Lente 15-D desta Micro-Ferramenta:**
 - **D1. Contratos e Regras:** frontmatter [name/description](#) no [SKILL.md](#); distribuída pela fonte única [componentes/compartilhado/skills/aidd-wizard/](#).
 - **D2. Input e Gatilhos:** gatilho pela [description](#) do frontmatter (nenhum arquivo em [comandos/](#) aponta para ela).
 - **D3. Raio de Impacto e Isolamento:** o [SKILL.md](#) não cita worktree nem isolamento.
 - **D4. Componentes e Fractalidade:** scripts: nenhum script próprio (roteiro em [SKILL.md](#)); MCPs citados: nenhum.

- ▶ **D5. Visão e Escopo:** Generate an interactive bash wizard that walks a human through steps only they can perform. Use when provisioning infrastructure (VPS, Cloudflare), setting up credentials or CI secrets, walking a third-party dashboard, or running a one-off migration or cutover. Use only for steps the agent cannot perform itself.
- ▶ **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#).
- ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
- ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo (sem motor próprio).
- ▶ **D9. O que o Estágio Entrega:** o que o roteiro do [SKILL.md](#) orienta produzir (ver D5); não há motor próprio que grave arquivo.
- ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
- ▶ **D11. Tratamento de Exceções e Fallback:** o [SKILL.md](#) cita `fallback`.
- ▶ **D12. Observabilidade e Frugalidade:** o [SKILL.md](#) cita `log`.
- ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
- ▶ **D14. Critério de Rejeição (Rollback):** o [SKILL.md](#) não cita rollback nem limpeza.
- ▶ **D15. Output Consolidado e Handoff:** o [SKILL.md](#) não cita handoff. Ainda sem laudo 15-D próprio.

44. Micro-Ferramenta: [aidd-xlsx](#)

- **Foto / Identidade:** [aidd-xlsx](#)
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-xlsx/SKILL.md`
- **O que Faz (Missão Única):** Use when creating, reading, editing, or validating spreadsheets (.xlsx, .xlsm, .csv, .tsv), including formulas, financial models, openpyxl scripts, pandas manipulation, or tabular data formatting.
- **Lente 15-D desta Micro-Ferramenta:**
 - ▶ **D1. Contratos e Regras:** frontmatter `name/description` no [SKILL.md](#); distribuída pela fonte única `componentes/compartilhado/skills/aidd-xlsx/`.
 - ▶ **D2. Input e Gatilhos:** gatilho pela `description` do frontmatter (nenhum arquivo em `comandos/` aponta para ela).
 - ▶ **D3. Raio de Impacto e Isolamento:** o [SKILL.md](#) não cita worktree nem isolamento.
 - ▶ **D4. Componentes e Fractalidade:** scripts: `recalc.py` em `scripts/`; MCPs citados: nenhum.
 - ▶ **D5. Visão e Escopo:** Use when creating, reading, editing, or validating spreadsheets (.xlsx, .xlsm, .csv, .tsv), including formulas, financial models, openpyxl scripts, pandas manipulation, or tabular data formatting.
 - ▶ **D6. O que o Estágio Faz:** executar o roteiro do [SKILL.md](#) e os scripts locais.
 - ▶ **D7. O que o Estágio Recebe:** o pedido do usuário ou do agente que a aciona.
 - ▶ **D8. O que o Estágio Processa:** roteiro seguido pelo modelo + scripts determinísticos.
 - ▶ **D9. O que o Estágio Entrega:** saída dos scripts locais e o que o roteiro orienta produzir (ver D5).
 - ▶ **D10. Orquestração e Topologia:** espelhada nos harnesses por `python ecossistema.py components sync --tipo todos`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** o [SKILL.md](#) cita `fallback`.
 - ▶ **D12. Observabilidade e Frugalidade:** o [SKILL.md](#) não cita log, telemetria nem `secoes/`.
 - ▶ **D13. Quality Gates (Portões):** `G_SKILL_ROT` confere caminhos e comandos citados. Roda automaticamente no commit via `.pre-commit-config.yaml`. Sem portão próprio.
 - ▶ **D14. Critério de Rejeição (Rollback):** o [SKILL.md](#) não cita rollback nem limpeza.
 - ▶ **D15. Output Consolidado e Handoff:** o [SKILL.md](#) não cita handoff. Ainda sem laudo 15-D próprio.

PARTE III: OS 65 GUARDIÕES INCORRUPÍVEIS (QUALITY GATES)

Na Festa

Imagine 65 cães de guarda robóticos sentados na saída da fábrica. Cada um tem um sensor diferente. Um cheira se tem segredo vazando, outro mede a espessura da parede, outro confere se tem botão quebrado e outro morde o pneu para ver se está furado. Se um único guarda latir (der exit 1), o portão de saída se tranca imediatamente e ninguém sai até consertar!

Na Casa (A Matriz dos 65 Quality Gates Canônicos)

Localização canônica: `gates/G_*.py`

Cada guarda também é lido pela **Lente 15-D**. Aqui a evidência vem do próprio código Python, por AST: flags do `argparse`, chamadas que gravam ou apagam arquivos, módulos do repositório importados, blocos `try`, saídas `print` e se o `.pre-commit-config.yaml` o chama. Um portão é de estágio único, então D6 a D9 descrevem uma passada só.

1. Guarda Incorruptível: `G_aidd_diagnose.py`

- **Foto / Identidade:** `G_aidd_diagnose.py`
- **Caminho no Disco:** `gates/G_aidd_diagnose.py`
- **Missão de Segurança:** Quality Gate Determinístico e Rótulo Honesto para a ferramenta `aidd-diagnose`.
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** flags `--relatorio`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código; executa subprocessos, cujos efeitos não entram nesta contagem.
 - **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - **D5. Visão e Escopo:** Quality Gate Determinístico e Rótulo Honesto para a ferramenta `aidd-diagnose`.
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - **D8. O que o Estágio Processa:** técnica: expressão regular, subprocesso.
 - **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - **D11. Tratamento de Exceções e Fallback:** 3 bloco(s) `try` no código.
 - **D12. Observabilidade e Frugalidade:** 4 saída(s) `print` para o terminal.
 - **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_aidd_diagnose.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

2. Guarda Incorruptível: `G_aidd_enterprise.py`

- **Foto / Identidade:** `G_aidd_enterprise.py`
- **Caminho no Disco:** `gates/G_aidd_enterprise.py`
- **Missão de Segurança:** Quality Gate Determinístico de contratos de componentes do aidd-enterprise
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** flags `--dir`, `--manifest`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - **D5. Visão e Escopo:** Quality Gate Determinístico de contratos de componentes do aidd-enterprise
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - **D8. O que o Estágio Processa:** técnica: JSON Schema, hash criptográfico.
 - **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - **D11. Tratamento de Exceções e Fallback:** 6 bloco(s) `try` no código.
 - **D12. Observabilidade e Frugalidade:** 3 saída(s) `print` para o terminal.
 - **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_aidd_enterprise.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

3. Guarda Incorruptível: `G_aidd_forge.py`

- **Foto / Identidade:** `G_aidd_forge.py`
- **Caminho no Disco:** `gates/G_aidd_forge.py`
- **Missão de Segurança:** Quality Gate Determinístico de bootstrap do aidd-forge (Lei #8 / Lei #13).
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** flags `--alvo`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - **D5. Visão e Escopo:** Quality Gate Determinístico de bootstrap do aidd-forge (Lei #8 / Lei #13).
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - **D8. O que o Estágio Processa:** técnica: AST Python, expressão regular.
 - **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - **D11. Tratamento de Exceções e Fallback:** 3 bloco(s) `try` no código.
 - **D12. Observabilidade e Frugalidade:** 3 saída(s) `print` para o terminal.

- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_aidd_forge.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

4. Guarda Incorrutível: `G_aidd_grill.py`

- **Foto / Identidade:** `G_aidd_grill.py`
- **Caminho no Disco:** `gates/G_aidd_grill.py`
- **Missão de Segurança:** Quality Gate Determinístico de Entrevista Socrática (Ticket 7 / D13 / DoD 6 / Leis #8, #9 e #13).
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--arquivo`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Quality Gate Determinístico de Entrevista Socrática (Ticket 7 / D13 / DoD 6 / Leis #8, #9 e #13).
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: leitura direta de arquivos.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 6 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_aidd_grill.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

5. Guarda Incorrutível: `G_aidd_spec.py`

- **Foto / Identidade:** `G_aidd_spec.py`
- **Caminho no Disco:** `gates/G_aidd_spec.py`
- **Missão de Segurança:** Quality Gate Determinístico de Especificação Técnica (Ticket 7 / D13 / DoD 6 / Leis #8, #9 e #13).
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--arquivo`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Quality Gate Determinístico de Especificação Técnica (Ticket 7 / D13 / DoD 6 / Leis #8, #9 e #13).
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: leitura direta de arquivos.

- ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
- ▶ **D12. Observabilidade e Frugalidade:** 6 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_aidd_spec.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

6. Guarda Incorrupível: `G_aidd_tdd.py`

- **Foto / Identidade:** `G_aidd_tdd.py`
- **Caminho no Disco:** `gates/G_aidd_tdd.py`
- **Missão de Segurança:** Quality Gate Determinístico de TDD (Ticket 7 / D13 / DoD 6 / Leis #8, #9 e #13).
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--sessao`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Quality Gate Determinístico de TDD (Ticket 7 / D13 / DoD 6 / Leis #8, #9 e #13).
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: leitura direta de arquivos.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 2 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 5 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_aidd_tdd.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

7. Guarda Incorrupível: `G_aidd_tickets.py`

- **Foto / Identidade:** `G_aidd_tickets.py`
- **Caminho no Disco:** `gates/G_aidd_tickets.py`
- **Missão de Segurança:** Quality Gate Determinístico de Decomposição de Tickets (Ticket 7 / D13 / DoD 6 / Leis #8, #9 e #13).
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--arquivo`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Quality Gate Determinístico de Decomposição de Tickets (Ticket 7 / D13 / DoD 6 / Leis #8, #9 e #13).

- ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
- ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
- ▶ **D8. O que o Estágio Processa:** técnica: leitura direta de arquivos.
- ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
- ▶ **D12. Observabilidade e Frugalidade:** 6 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_aidd_tickets.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

8. Guarda Incorruptível: `G_amelhoria.py`

- **Foto / Identidade:** `G_amelhoria.py`
- **Caminho no Disco:** `gates/G_amelhoria.py`
- **Missão de Segurança:** Quality Gate Determinístico de Rótulo Honesto para aidd-melhoria.
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--relatorio`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Quality Gate Determinístico de Rótulo Honesto para aidd-melhoria.
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: expressão regular.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 2 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 4 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_amelhoria.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

9. Guarda Incorruptível: `G_ANT_LOCKIN_LEGADO.py`

- **Foto / Identidade:** `G_ANT_LOCKIN_LEGADO.py`
- **Caminho no Disco:** `gates/G_ANT_LOCKIN_LEGADO.py`
- **Missão de Segurança:** Varre entrega/legado por resíduo lovable|supabase|firebase e dirs `.lovable/`.
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).

- ▶ **D5. Visão e Escopo:** Varre entrega/legado por resíduo lovable|supabase|firebase e dirs `./lovable/`,
- ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
- ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
- ▶ **D8. O que o Estágio Processa:** técnica: leitura direta de arquivos.
- ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
- ▶ **D12. Observabilidade e Frugalidade:** 7 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_ant_lockin_legacy.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

10. Guarda Incorrúptível: `G_ARQUITETURA_DELIVERABLE.py`

- **Foto / Identidade:** `G_ARQUITETURA_DELIVERABLE.py`
- **Caminho no Disco:** `gates/G_ARQUITETURA_DELIVERABLE.py`
- **Missão de Segurança:** Gate estatico via AST que valida conformidade com Clean Architecture e DDD
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Gate estatico via AST que valida conformidade com Clean Architecture e DDD
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
 - ▶ **D8. O que o Estágio Processa:** técnica: AST Python.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 2 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 21 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_arquitetura_deliverable.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

11. Guarda Incorrúptível: `G_CLI_HELP_CONSISTENCIA.py`

- **Foto / Identidade:** `G_CLI_HELP_CONSISTENCIA.py`
- **Caminho no Disco:** `gates/G_CLI_HELP_CONSISTENCIA.py`
- **Missão de Segurança:** Detecta divergência entre flags de CLI realmente definidas via
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.

- ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
- ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
- ▶ **D5. Visão e Escopo:** Detecta divergência entre flags de CLI realmente definidas via
- ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
- ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
- ▶ **D8. O que o Estágio Processa:** técnica: AST Python, expressão regular.
- ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
- ▶ **D12. Observabilidade e Frugalidade:** 12 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_cli_help_consistencia.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

12. Guarda IncorrupΤível: `G_COMPONENTE_AGNOSTICO.py`

- **Foto / Identidade:** `G_COMPONENTE_AGNOSTICO.py`
- **Caminho no Disco:** `gates/G_COMPONENTE_AGNOSTICO.py`
- **Missão de Segurança:** Audita se todo componente novo ou modificado (detectado via git diff / status)
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--base`, `--todos`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código; executa subprocessos, cujos efeitos não entram nesta contagem.
 - ▶ **D4. Componentes e Fractalidade:** importa módulos do repositório: `gestor_componentes`
 - ▶ **D5. Visão e Escopo:** Audita se todo componente novo ou modificado (detectado via git diff / status)
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: subprocesso.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 2 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 19 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_componente_agnostico.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

13. Guarda IncorrupΤível: `G_CONTRACT_ROT.py`

- **Foto / Identidade:** `G_CONTRACT_ROT.py`
- **Caminho no Disco:** `gates/G_CONTRACT_ROT.py`
- **Missão de Segurança:** Portão determinístico de prevenção de Contract Rot.
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).

- ▶ **D2. Input e Gatilhos:** flags `--committed-spec`, `--projeto`, `--url`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
- ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código; executa subprocessos, cujos efeitos não entram nesta contagem.
- ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
- ▶ **D5. Visão e Escopo:** Portão determinístico de prevenção de Contract Rot.
- ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
- ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
- ▶ **D8. O que o Estágio Processa:** técnica: subprocesso.
- ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- ▶ **D11. Tratamento de Exceções e Fallback:** 6 bloco(s) `try` no código.
- ▶ **D12. Observabilidade e Frugalidade:** 26 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_contract_rot.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

14. Guarda Incorruptível: `G_DEPENDENCIAS_PIN_HASH.py`

- **Foto / Identidade:** `G_DEPENDENCIAS_PIN_HASH.py`
- **Caminho no Disco:** `gates/G_DEPENDENCIAS_PIN_HASH.py`
- **Missão de Segurança:** PLAN-0018 fase 04 (pin-exato-e-hashes-requirements-lockfile — SEC-4):
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** PLAN-0018 fase 04 (pin-exato-e-hashes-requirements-lockfile — SEC-4)
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
 - ▶ **D8. O que o Estágio Processa:** técnica: expressão regular.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** nenhum bloco `try`: erro inesperado derruba o portão (e o commit reprova).
 - ▶ **D12. Observabilidade e Frugalidade:** 15 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_dependencias_pin_hash.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

15. Guarda Incorruptível: `G_DETERMINISMO_LEI_1.py`

- **Foto / Identidade:** `G_DETERMINISMO_LEI_1.py`
- **Caminho no Disco:** `gates/G_DETERMINISMO_LEI_1.py`

- **Missão de Segurança:** Auditoria de Determinismo Mecânico: Bloqueia o uso de SDKs e chamadas de LLM
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--repo-root`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Auditoria de Determinismo Mecânico: Bloqueia o uso de SDKs e chamadas de LLM
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: AST Python.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 2 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 16 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_determinismo_lei_1.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

16. Guarda Incorruptível: `G_DISCIPLINA_TESTE_FERRAMENTA.py`

- **Foto / Identidade:** `G_DISCIPLINA_TESTE_FERRAMENTA.py`
- **Caminho no Disco:** `gates/G_DISCIPLINA_TESTE_FERRAMENTA.py`
- **Missão de Segurança:** Auditoria de Disciplina de Teste e Validação de Ferramentas.
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--files`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código; executa subprocessos, cujos efeitos não entram nesta contagem.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Auditoria de Disciplina de Teste e Validação de Ferramentas.
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: subprocesso.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** nenhum bloco `try`: erro inesperado derruba o portão (e o commit reprova).
 - ▶ **D12. Observabilidade e Frugalidade:** 17 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_disciplina_teste_ferramenta.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

17. Guarda Incorruptível: `G_DISPATCH_PIPELINE_VSA.py`

- **Foto / Identidade:** `G_DISPATCH_PIPELINE_VSA.py`
- **Caminho no Disco:** `gates/G_DISPATCH_PIPELINE_VSA.py`
- **Missão de Segurança:** Portão determinístico de validação formal de manifestos de despacho topológico
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** flags `--manifesto`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - **D5. Visão e Escopo:** Portão determinístico de validação formal de manifestos de despacho topológico
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - **D8. O que o Estágio Processa:** técnica: expressão regular, JSON Schema.
 - **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - **D11. Tratamento de Exceções e Fallback:** 4 bloco(s) `try` no código.
 - **D12. Observabilidade e Frugalidade:** 25 saída(s) `print` para o terminal.
 - **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_dispatch_pipeline_vsa.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

18. Guarda Incorruptível: `G_DOCS_ROT.py`

- **Foto / Identidade:** `G_DOCS_ROT.py`
- **Caminho no Disco:** `gates/G_DOCS_ROT.py`
- **Missão de Segurança:** Verifica:
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - **D5. Visão e Escopo:** Verifica
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** o repositório inteiro.
 - **D8. O que o Estágio Processa:** técnica: expressão regular.
 - **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
 - **D12. Observabilidade e Frugalidade:** 4 saída(s) `print` para o terminal.
 - **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_docs_rot.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.

- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

19. Guarda IncorrupΤível: `G_DRIFT_ANALYZER.py`

- **Foto / Identidade:** `G_DRIFT_ANALYZER.py`
- **Caminho no Disco:** `gates/G_DRIFT_ANALYZER.py`
- **Missão de Segurança:** Detecta redundâncias estruturais e duplicidade de assinaturas de funções
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--strict`, `--target`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Detecta redundâncias estruturais e duplicidade de assinaturas de funções
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: AST Python, hash criptográfico.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 6 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_drift_analyzer.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

20. Guarda IncorrupΤível: `G_DRIFT_NUCLEO_COMPARTILHADO.py`

- **Foto / Identidade:** `G_DRIFT_NUCLEO_COMPARTILHADO.py`
- **Caminho no Disco:** `gates/G_DRIFT_NUCLEO_COMPARTILHADO.py`
- **Missão de Segurança:** Detecta divergência silenciosa entre pares de diretórios que nasceram da
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** grava ou remove arquivos (`open(w)`).
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Detecta divergência silenciosa entre pares de diretórios que nasceram da
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
 - ▶ **D8. O que o Estágio Processa:** técnica: hash criptográfico.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** nenhum bloco `try`: erro inesperado derruba o portão (e o commit reprova).

- ▶ **D12. Observabilidade e Frugalidade:** 16 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_drift_nucleo_compartilhado.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

21. Guarda Incorrúptível: `G_ECOSSISTEMA_INTEGRIDADE.py`

- **Foto / Identidade:** `G_ECOSSISTEMA_INTEGRIDADE.py`
- **Caminho no Disco:** `gates/G_ECOSSISTEMA_INTEGRIDADE.py`
- **Missão de Segurança:** Validação determinística de integridade do meta-repositório `ecossistema-aidd`.
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Validação determinística de integridade do meta-repositório `ecossistema-aidd`.
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
 - ▶ **D8. O que o Estágio Processa:** técnica: AST Python, expressão regular.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo `pre-commit`.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 22 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_ecossistema_integridade.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

22. Guarda Incorrúptível: `G_ENV_ROT.py`

- **Foto / Identidade:** `G_ENV_ROT.py`
- **Caminho no Disco:** `gates/G_ENV_ROT.py`
- **Missão de Segurança:** Quality Gate determinístico de prevenção a Config & Environment Rot.
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--env-file`, `--root-dir`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Quality Gate determinístico de prevenção a Config & Environment Rot.
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: AST Python, expressão regular.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).

- ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- ▶ **D11. Tratamento de Exceções e Fallback:** 4 bloco(s) `try` no código.
- ▶ **D12. Observabilidade e Frugalidade:** 34 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_env_rot.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

23. Guarda Incorrupível: `G_ESCRITOR_ATOMICO.py`

- **Foto / Identidade:** `G_ESCRITOR_ATOMICO.py`
- **Caminho no Disco:** `gates/G_ESCRITOR_ATOMICO.py`
- **Missão de Segurança:** Verificação determinística (AST) de que os pontos críticos de gravação de
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Verificação determinística (AST) de que os pontos críticos de gravação de
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
 - ▶ **D8. O que o Estágio Processa:** técnica: AST Python.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 13 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_escritor_atomico.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

24. Guarda Incorrupível: `G ESTRUTURA_ESTADO.py`

- **Foto / Identidade:** `G ESTRUTURA_ESTADO.py`
- **Caminho no Disco:** `gates/G ESTRUTURA_ESTADO.py`
- **Missão de Segurança:** Auditoria de Persistência Estruturada de Estado de Orquestração.
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--check-artifact`, `--type`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Auditoria de Persistência Estruturada de Estado de Orquestração.
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.

- ▶ **D8. O que o Estágio Processa:** técnica: leitura direta de arquivos.
- ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- ▶ **D11. Tratamento de Exceções e Fallback:** 6 bloco(s) `try` no código.
- ▶ **D12. Observabilidade e Frugalidade:** 21 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_estrutura_estado.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

25. Guarda IncorrupΤível: `G_FRONTIERA_FERRAMENTAS.py`

- **Foto / Identidade:** `G_FRONTIERA_FERRAMENTAS.py`
- **Caminho no Disco:** `gates/G_FRONTIERA_FERRAMENTAS.py`
- **Missão de Segurança:** Gate de fronteira das 8 ferramentas em modo aviso (Ticket 6).
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--modo`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código; executa subprocessos, cujos efeitos não entram nesta contagem.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Gate de fronteira das 8 ferramentas em modo aviso (Ticket 6).
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: expressão regular, subprocesso, hash criptográfico.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 2 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 14 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_frontera_ferramentas.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

26. Guarda IncorrupΤível: `G_FRONTEND_LAYERS.py`

- **Foto / Identidade:** `G_FRONTEND_LAYERS.py`
- **Caminho no Disco:** `gates/G_FRONTEND_LAYERS.py`
- **Missão de Segurança:** Validação estática de separação de camadas nas entregas e templates de Frontend.
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Validação estática de separação de camadas nas entregas e templates de Frontend.

- ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
- ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
- ▶ **D8. O que o Estágio Processa:** técnica: expressão regular.
- ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
- ▶ **D12. Observabilidade e Frugalidade:** 10 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_frontend_layers.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

27. Guarda IncorrupΤível: `G_GESTOR_SESSOES.py`

- **Foto / Identidade:** `G_GESTOR_SESSOES.py`
- **Caminho no Disco:** `gates/G_GESTOR_SESSOES.py`
- **Missão de Segurança:** Valida a integridade determinística do subsistema de rastreamento de sessões:
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código; executa subprocessos, cujos efeitos não entram nesta contagem.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Valida a integridade determinística do subsistema de rastreamento de sessões
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
 - ▶ **D8. O que o Estágio Processa:** técnica: AST Python, subprocesso.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 12 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_gestor_sesoes.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

28. Guarda IncorrupΤível: `G_GRAPH_FIRST.py`

- **Foto / Identidade:** `G_GRAPH_FIRST.py`
- **Caminho no Disco:** `gates/G_GRAPH_FIRST.py`
- **Missão de Segurança:** Lê o histórico do agente que trabalhou numa worktree e reprova se ele pesquisou
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--desde`, `--worktree`. Não está no `.pre-commit-config.yaml`: roda sob demanda (`python gates/G_GRAPH_FIRST.py`).

- ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
- ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
- ▶ **D5. Visão e Escopo:** Lê o histórico do agente que trabalhou numa worktree e reprova se ele pesquisou
- ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
- ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
- ▶ **D8. O que o Estágio Processa:** técnica: expressão regular.
- ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- ▶ **D10. Orquestração e Topologia:** acionado só quando alguém o chama (não bloqueia o commit sozinho).
- ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
- ▶ **D12. Observabilidade e Frugalidade:** 3 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_graph_first.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` só bloqueia quando o portão é chamado.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

29. Guarda Incorrupível: `G_HADOLINT.py`

- **Foto / Identidade:** `G_HADOLINT.py`
- **Caminho no Disco:** `gates/G_HADOLINT.py`
- **Missão de Segurança:** Audita estaticamente todos os Dockerfiles do repositório (e os gerados pelas
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código; executa subprocessos, cujos efeitos não entram nesta contagem.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Audita estaticamente todos os Dockerfiles do repositório (e os gerados pelas
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
 - ▶ **D8. O que o Estágio Processa:** técnica: subprocesso.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 3 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 23 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_hadolint.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

30. Guarda Incorrupível: `G_HANDOFF_MELHORIA.py`

- **Foto / Identidade:** `G_HANDOFF_MELHORIA.py`
- **Caminho no Disco:** `gates/G_HANDOFF_MELHORIA.py`
- **Missão de Segurança:** Portão de transição melhoria → plan consumido pelo orquestrador.

- **Lente 15-D deste Quality Gate:**

- ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
- ▶ **D2. Input e Gatilhos:** flags `--handoff`, `--repo-root`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
- ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
- ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
- ▶ **D5. Visão e Escopo:** Portão de transição melhoria → plan consumido pelo orquestrador.
- ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
- ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
- ▶ **D8. O que o Estágio Processa:** técnica: leitura direta de arquivos.
- ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- ▶ **D11. Tratamento de Exceções e Fallback:** nenhum bloco `try`: erro inesperado derruba o portão (e o commit reprova).
- ▶ **D12. Observabilidade e Frugalidade:** 3 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_handoff_melhoria.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

31. Guarda Incorrupível: `G_HARNESS_COMPAT.py`

- **Foto / Identidade:** `G_HARNESS_COMPAT.py`
- **Caminho no Disco:** `gates/G_HARNESS_COMPAT.py`
- **Missão de Segurança:** Materializa o gate `G_HARNESS_COMPAT` que o plano de execução original
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** importa módulos do repositório: `gestor_componentes`
 - ▶ **D5. Visão e Escopo:** Materializa o gate `G_HARNESS_COMPAT` que o plano de execução original
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
 - ▶ **D8. O que o Estágio Processa:** técnica: expressão regular.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** nenhum bloco `try`: erro inesperado derruba o portão (e o commit reprova).
 - ▶ **D12. Observabilidade e Frugalidade:** 20 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_harness_compat.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

32. Guarda IncorrupΤível: `G_HONESTIDADE_ROTULO.py`

- **Foto / Identidade:** `G_HONESTIDADE_ROTULO.py`
- **Caminho no Disco:** `gates/G_HONESTIDADE_ROTULO.py`
- **Missão de Segurança:** Verificacao mecanica da Regra de Ouro #9 (AGENTS.md §2, "Honestidade de
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - **D5. Visão e Escopo:** Verificacao mecanica da Regra de Ouro #9 (AGENTS.md §2, "Honestidade de
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** o repositório inteiro.
 - **D8. O que o Estágio Processa:** técnica: AST Python.
 - **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
 - **D12. Observabilidade e Frugalidade:** 12 saída(s) `print` para o terminal.
 - **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_honestidade_rotulo.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

33. Guarda IncorrupΤível: `G_IDIOMA_LEI_4.py`

- **Foto / Identidade:** `G_IDIOMA_LEI_4.py`
- **Caminho no Disco:** `gates/G_IDIOMA_LEI_4.py`
- **Missão de Segurança:** Verificação mecânica determinística da Lei #4 (AGENTS.md §2, "Extreme Token
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** flags `--caminho`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - **D5. Visão e Escopo:** Verificação mecânica determinística da Lei #4 (AGENTS.md §2, "Extreme Token
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - **D8. O que o Estágio Processa:** técnica: expressão regular.
 - **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
 - **D12. Observabilidade e Frugalidade:** 12 saída(s) `print` para o terminal.

- **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_idioma_lei_4.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

34. Guarda Incorruptível: `G_INFRA_COMPOSE.py`

- **Foto / Identidade:** `G_INFRA_COMPOSE.py`
- **Caminho no Disco:** `gates/G_INFRA_COMPOSE.py`
- **Missão de Segurança:** Audita a integridade estática, sintática, de segurança e topológica dos arquivos
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código; executa subprocessos, cujos efeitos não entram nesta contagem.
 - **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - **D5. Visão e Escopo:** Audita a integridade estática, sintática, de segurança e topológica dos arquivos
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** o repositório inteiro.
 - **D8. O que o Estágio Processa:** técnica: expressão regular, subprocesso, YAML.
 - **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - **D11. Tratamento de Exceções e Fallback:** 7 bloco(s) `try` no código.
 - **D12. Observabilidade e Frugalidade:** 35 saída(s) `print` para o terminal.
 - **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_infra_compose.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

35. Guarda Incorruptível: `G_ISOLATION_AUDIT.py`

- **Foto / Identidade:** `G_ISOLATION_AUDIT.py`
- **Caminho no Disco:** `gates/G_ISOLATION_AUDIT.py`
- **Missão de Segurança:** Auditoria estática de isolamento de fatias verticais (Vertical Slice Architecture - VSA).
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - **D5. Visão e Escopo:** Auditoria estática de isolamento de fatias verticais (Vertical Slice Architecture - VSA).
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** o repositório inteiro.
 - **D8. O que o Estágio Processa:** técnica: AST Python.

- ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
- ▶ **D12. Observabilidade e Frugalidade:** 11 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_isolation_audit.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

36. Guarda Incorrupível: `G_LAYOUT_ENTREGA.py`

- **Foto / Identidade:** `G_LAYOUT_ENTREGA.py`
- **Caminho no Disco:** `gates/G_LAYOUT_ENTREGA.py`
- **Missão de Segurança:** Bloqueia entrega do usuário gerada sob `<clone>/projetos/` quando existe
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Bloqueia entrega do usuário gerada sob `<clone>/projetos/` quando existe
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
 - ▶ **D8. O que o Estágio Processa:** técnica: expressão regular.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** nenhum bloco `try`: erro inesperado derruba o portão (e o commit reprova).
 - ▶ **D12. Observabilidade e Frugalidade:** 7 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_layout_entrega.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

37. Guarda Incorrupível: `G_LEI_DECLARA_PORTAO.py`

- **Foto / Identidade:** `G_LEI_DECLARA_PORTAO.py`
- **Caminho no Disco:** `gates/G_LEI_DECLARA_PORTAO.py`
- **Missão de Segurança:** Meta-Quality Gate que audita e bloqueia se qualquer Lei Inviolável em `AGENTS.md`
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--agents-file`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Meta-Quality Gate que audita e bloqueia se qualquer Lei Inviolável em `AGENTS.md`

- ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
- ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
- ▶ **D8. O que o Estágio Processa:** técnica: expressão regular.
- ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
- ▶ **D12. Observabilidade e Frugalidade:** 19 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_lei_declara_portao.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

38. Guarda IncorrupΤível: `G_LIVRO_EVIDENCIA.py`

- **Foto / Identidade:** `G_LIVRO_EVIDENCIA.py`
- **Caminho no Disco:** `gates/G_LIVRO_EVIDENCIA.py`
- **Missão de Segurança:** Audita deterministicamente o livro-texto gerado para um projeto, impedindo que
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--json`, `--livro`, `--projeto`. Registrado no `.pre-commit-config.yaml` com `stages: [manual]` (roda só quando chamado).
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Audita deterministicamente o livro-texto gerado para um projeto, impedindo que
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: expressão regular.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado só quando alguém o chama (não bloqueia o commit sozinho).
 - ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 20 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_livro_evidencia.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` só bloqueia quando o portão é chamado.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

39. Guarda IncorrupΤível: `G_LLM_PROMPT_SHIELD.py`

- **Foto / Identidade:** `G_LLM_PROMPT_SHIELD.py`
- **Caminho no Disco:** `gates/G_LLM_PROMPT_SHIELD.py`
- **Missão de Segurança:** Auditoria de Blindagem e Sanitização Anti-Prompt Injection em Clientes LLM.
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.

- ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
- ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
- ▶ **D5. Visão e Escopo:** Auditoria de Blindagem e Sanitização Anti-Prompt Injection em Clientes LLM.
- ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
- ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
- ▶ **D8. O que o Estágio Processa:** técnica: AST Python.
- ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
- ▶ **D12. Observabilidade e Frugalidade:** 9 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_llm_prompt_shield.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

40. Guarda Incorruptível: `G_mapa_pecas.py`

- **Foto / Identidade:** `G_mapa_pecas.py`
- **Caminho no Disco:** `gates/G_mapa_pecas.py`
- **Missão de Segurança:** Quality Gate Determinístico de Integridade do Mapa de Peças e Catálogo.
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--achados`, `--catalogo`, `--mapas-dir`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Quality Gate Determinístico de Integridade do Mapa de Peças e Catálogo.
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: leitura direta de arquivos.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 3 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 8 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_mapa_pecas.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

41. Guarda Incorruptível: `G_MIGRATION_ROT.py`

- **Foto / Identidade:** `G_MIGRATION_ROT.py`
- **Caminho no Disco:** `gates/G_MIGRATION_ROT.py`
- **Missão de Segurança:** Quality Gate determinístico de prevenção a Migration Rot.
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).

- ▶ **D2. Input e Gatilhos:** flags `--target`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
- ▶ **D3. Raio de Impacto e Isolamento:** grava ou remove arquivos (`os.remove`); executa subprocessos, cujos efeitos não entram nesta contagem.
- ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
- ▶ **D5. Visão e Escopo:** Quality Gate determinístico de prevenção a Migration Rot.
- ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
- ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
- ▶ **D8. O que o Estágio Processa:** técnica: AST Python, subprocesso.
- ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- ▶ **D11. Tratamento de Exceções e Fallback:** 5 bloco(s) `try` no código.
- ▶ **D12. Observabilidade e Frugalidade:** 27 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_migration_rot.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

42. Guarda Incorrúptível: `G_NOVE_CAMADAS_MERCADO.py`

- **Foto / Identidade:** `G_NOVE_CAMADAS_MERCADO.py`
- **Caminho no Disco:** `gates/G_NOVE_CAMADAS_MERCADO.py`
- **Missão de Segurança:** Portão determinístico de validação arquitetural das 9 Camadas de Mercado
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--target`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Portão determinístico de validação arquitetural das 9 Camadas de Mercado
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: leitura direta de arquivos.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 5 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 11 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_nove_camadas_mercado.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

43. Guarda Incorrúptível: `G_ORQUESTRADOR_SINCRONO.py`

- **Foto / Identidade:** `G_ORQUESTRADOR_SINCRONO.py`
- **Caminho no Disco:** `gates/G_ORQUESTRADOR_SINCRONO.py`
- **Missão de Segurança:** Quality Gate: `G_ORQUESTRADOR_SINCRONO`

- **Lente 15-D deste Quality Gate:**

- **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
- **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
- **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código; executa subprocessos, cujos efeitos não entram nesta contagem.
- **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
- **D5. Visão e Escopo:** Quality Gate: G_ORQUESTADOR_SINCRONO
- **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
- **D7. O que o Estágio Recebe:** o repositório inteiro.
- **D8. O que o Estágio Processa:** técnica: subprocesso.
- **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- **D11. Tratamento de Exceções e Fallback:** 2 bloco(s) `try` no código.
- **D12. Observabilidade e Frugalidade:** 15 saída(s) `print` para o terminal.
- **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_orquestrador_sincrono.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

44. Guarda Incorrupível: `G_PACOTE_CORE.py`

- **Foto / Identidade:** `G_PACOTE_CORE.py`
- **Caminho no Disco:** `gates/G_PACOTE_CORE.py`
- **Missão de Segurança:** Falha se a árvore de distribuição (export / pacote usuário) contiver artefatos
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - **D5. Visão e Escopo:** Falha se a árvore de distribuição (export / pacote usuário) contiver artefatos
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** o repositório inteiro.
 - **D8. O que o Estágio Processa:** técnica: leitura direta de arquivos.
 - **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - **D11. Tratamento de Exceções e Fallback:** nenhum bloco `try`: erro inesperado derruba o portão (e o commit reprova).
 - **D12. Observabilidade e Frugalidade:** 6 saída(s) `print` para o terminal.
 - **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_pacote_core.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

45. Guarda IncorrupΤίvel: `G_PIPELINE_HANDOFF.py`

- **Foto / Identidade:** `G_PIPELINE_HANDOFF.py`
- **Caminho no Disco:** `gates/G_PIPELINE_HANDOFF.py`
- **Missão de Segurança:** Portão determinístico de validação formal de manifestos de handoff de execução
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** flags `--manifesto`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - **D5. Visão e Escopo:** Portão determinístico de validação formal de manifestos de handoff de execução
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - **D8. O que o Estágio Processa:** técnica: expressão regular, JSON Schema.
 - **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - **D11. Tratamento de Exceções e Fallback:** 4 bloco(s) `try` no código.
 - **D12. Observabilidade e Frugalidade:** 20 saída(s) `print` para o terminal.
 - **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_pipeline_handoff.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

46. Guarda IncorrupΤίvel: `G_PORTAO_PROVA_QUE_MORDE.py`

- **Foto / Identidade:** `G_PORTAO_PROVA_QUE_MORDE.py`
- **Caminho no Disco:** `gates/G_PORTAO_PROVA_QUE_MORDE.py`
- **Missão de Segurança:** Meta-Quality Gate que audita e bloqueia qualquer gate que seja entregue
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código; executa subprocessos, cujos efeitos não entram nesta contagem.
 - **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - **D5. Visão e Escopo:** Meta-Quality Gate que audita e bloqueia qualquer gate que seja entregue
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** o repositório inteiro.
 - **D8. O que o Estágio Processa:** técnica: AST Python, expressão regular, subprocesso.
 - **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - **D11. Tratamento de Exceções e Fallback:** 3 bloco(s) `try` no código.
 - **D12. Observabilidade e Frugalidade:** 27 saída(s) `print` para o terminal.

- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_portao_prova_que_morde.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

47. Guarda Incorruptível: `G_PROTOCOL_FALLBACK.py`

- **Foto / Identidade:** `G_PROTOCOL_FALLBACK.py`
- **Caminho no Disco:** `gates/G_PROTOCOL_FALLBACK.py`
- **Missão de Segurança:** Auditoria de Paridade e Fallback REST vs Model Context Protocol (MCP).
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Auditoria de Paridade e Fallback REST vs Model Context Protocol (MCP).
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
 - ▶ **D8. O que o Estágio Processa:** técnica: leitura direta de arquivos.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 9 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_protocol_fallback.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

48. Guarda Incorruptível: `G_PROTOTYPE_REWRITE.py`

- **Foto / Identidade:** `G_PROTOTYPE_REWRITE.py`
- **Caminho no Disco:** `gates/G_PROTOTYPE_REWRITE.py`
- **Missão de Segurança:** Garante o isolamento determinístico de protótipos em sandbox/ e impede a
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--target`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Garante o isolamento determinístico de protótipos em sandbox/ e impede a
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: AST Python.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).

- ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- ▶ **D11. Tratamento de Exceções e Fallback:** 2 bloco(s) `try` no código.
- ▶ **D12. Observabilidade e Frugalidade:** 8 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_prototype_rewrite.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

49. Guarda Incorrúptível: `G_PROVA_SKILLS_POCOCK.py`

- **Foto / Identidade:** `G_PROVA_SKILLS_POCOCK.py`
- **Caminho no Disco:** `gates/G_PROVA_SKILLS_POCOCK.py`
- **Missão de Segurança:** Prova de uso real das skills alteradas no ciclo skills-pocock/ciclo-01.
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--artefatos`, `--manter`, `--skill`. Registrado no `.pre-commit-config.yaml` com `stages: [manual]` (roda só quando chamado).
 - ▶ **D3. Raio de Impacto e Isolamento:** grava ou remove arquivos (`copy2`, `copytree`, `mkdir`, `write_text`); executa subprocessos, cujos efeitos não entram nesta contagem.
 - ▶ **D4. Componentes e Fractalidade:** importa módulos do repositório: `compilador_tickets_plano`
 - ▶ **D5. Visão e Escopo:** Prova de uso real das skills alteradas no ciclo skills-pocock/ciclo-01.
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: expressão regular, subprocesso.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado só quando alguém o chama (não bloqueia o commit sozinho).
 - ▶ **D11. Tratamento de Exceções e Fallback:** 3 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 6 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_prova_skills_pocock.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` só bloqueia quando o portão é chamado.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

50. Guarda Incorrúptível: `G_QUARTETO_SINE_QUA_NON.py`

- **Foto / Identidade:** `G_QUARTETO_SINE_QUA_NON.py`
- **Caminho no Disco:** `gates/G_QUARTETO_SINE_QUA_NON.py`
- **Missão de Segurança:** Portão determinístico de auditoria do Quarteto Sine Qua Non Dinâmico.
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--target`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Portão determinístico de auditoria do Quarteto Sine Qua Non Dinâmico.
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: expressão regular.

- **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- **D11. Tratamento de Exceções e Fallback:** 3 bloco(s) `try` no código.
- **D12. Observabilidade e Frugalidade:** 27 saída(s) `print` para o terminal.
- **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_quarteto_sine_qua_non.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

51. Guarda Incorruptível: `G_RESUMO_USUARIO.py`

- **Foto / Identidade:** `G_RESUMO_USUARIO.py`
- **Caminho no Disco:** `gates/G_RESUMO_USUARIO.py`
- **Missão de Segurança:** Exige, quando existe raiz de entrega com `README-USUARIO.md`, também
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - **D5. Visão e Escopo:** Exige, quando existe raiz de entrega com `README-USUARIO.md`, também
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** o repositório inteiro.
 - **D8. O que o Estágio Processa:** técnica: leitura direta de arquivos.
 - **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - **D11. Tratamento de Exceções e Fallback:** nenhum bloco `try`: erro inesperado derruba o portão (e o commit reprova).
 - **D12. Observabilidade e Frugalidade:** 7 saída(s) `print` para o terminal.
 - **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_resumo_usuario.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

52. Guarda Incorruptível: `G_SAIDA_BINARIA.py`

- **Foto / Identidade:** `G_SAIDA_BINARIA.py`
- **Caminho no Disco:** `gates/G_SAIDA_BINARIA.py`
- **Missão de Segurança:** Auditoria de Qualidade Binária: Verifica deterministicamente via AST que todos
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).

- **D5. Visão e Escopo:** Auditoria de Qualidade Binária: Verifica deterministicamente via AST que todos
- **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
- **D7. O que o Estágio Recebe:** o repositório inteiro.
- **D8. O que o Estágio Processa:** técnica: AST Python.
- **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
- **D12. Observabilidade e Frugalidade:** 14 saída(s) `print` para o terminal.
- **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_saida_binaria.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

53. Guarda Incorrūptível: `G_SEGREDOS.py`

- **Foto / Identidade:** `G_SEGREDOS.py`
- **Caminho no Disco:** `gates/G_SEGREDOS.py`
- **Missão de Segurança:** Escaneia arquivos em busca de credenciais hardcoded, delegando a varredura
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** grava ou remove arquivos (`open(w)`); executa subprocessos, cujos efeitos não entram nesta contagem.
 - **D4. Componentes e Fractalidade:** importa módulos do repositório: `_escopo_commit`
 - **D5. Visão e Escopo:** Escaneia arquivos em busca de credenciais hardcoded, delegando a varredura
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** o repositório inteiro.
 - **D8. O que o Estágio Processa:** técnica: subprocesso.
 - **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - **D11. Tratamento de Exceções e Fallback:** 5 bloco(s) `try` no código.
 - **D12. Observabilidade e Frugalidade:** 18 saída(s) `print` para o terminal.
 - **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_segredos.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

54. Guarda Incorrūptível: `G_SKILL_FORMATO.py`

- **Foto / Identidade:** `G_SKILL_FORMATO.py`
- **Caminho no Disco:** `gates/G_SKILL_FORMATO.py`
- **Missão de Segurança:** Confere o formato de cada skill da fonte unica contra
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** flags `--apenas`, `--aviso`, `--raiz`, `--secoes-estritas`. Roda automaticamente no commit via `.pre-commit-config.yaml`.

- **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
- **D4. Componentes e Fractalidade:** importa módulos do repositório: `gestor_dependencias`
- **D5. Visão e Escopo:** Confere o formato de cada skill da fonte unica contra
- **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
- **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
- **D8. O que o Estágio Processa:** técnica: expressão regular, YAML.
- **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
- **D12. Observabilidade e Frugalidade:** 9 saída(s) `print` para o terminal.
- **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_skill_formato.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

55. Guarda Incorruptível: `G_SKILL_ROT.py`

- **Foto / Identidade:** `G_SKILL_ROT.py`
- **Caminho no Disco:** `gates/G_SKILL_ROT.py`
- **Missão de Segurança:** Quality Gate determinístico de prevenção a Skill & Tool Rot.
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** flags `--repo-root`, `--skills-dir`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - **D4. Componentes e Fractalidade:** importa módulos do repositório: `gestor_dependencias`
 - **D5. Visão e Escopo:** Quality Gate determinístico de prevenção a Skill & Tool Rot.
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - **D8. O que o Estágio Processa:** técnica: expressão regular.
 - **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
 - **D12. Observabilidade e Frugalidade:** 20 saída(s) `print` para o terminal.
 - **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_skill_rot.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

56. Guarda Incorruptível: `G_STACK_PADRAO_OURO.py`

- **Foto / Identidade:** `G_STACK_PADRAO_OURO.py`
- **Caminho no Disco:** `gates/G_STACK_PADRAO_OURO.py`
- **Missão de Segurança:** Portão determinístico do Padrão-Ouro de Stack Tecnológica (Lei Canônica #11).
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).

- ▶ **D2. Input e Gatilhos:** flags `--target`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
- ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
- ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
- ▶ **D5. Visão e Escopo:** Portão determinístico do Padrão-Ouro de Stack Tecnológica (Lei Canônica #11).
- ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
- ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
- ▶ **D8. O que o Estágio Processa:** técnica: expressão regular.
- ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
- ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- ▶ **D11. Tratamento de Exceções e Fallback:** 7 bloco(s) `try` no código.
- ▶ **D12. Observabilidade e Frugalidade:** 24 saída(s) `print` para o terminal.
- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_stack_padrao_ouro.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

57. Guarda Incorruptível: `G_SUPPLY_CHAIN.py`

- **Foto / Identidade:** `G_SUPPLY_CHAIN.py`
- **Caminho no Disco:** `gates/G_SUPPLY_CHAIN.py`
- **Missão de Segurança:** PLAN-0031 fase 01: Verificação da integridade da cadeia de suprimentos e
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código; executa subprocessos, cujos efeitos não entram nesta contagem.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** PLAN-0031 fase 01: Verificação da integridade da cadeia de suprimentos e
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
 - ▶ **D8. O que o Estágio Processa:** técnica: expressão regular, subprocesso.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 3 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 13 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_supply_chain.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

58. Guarda Incorruptível: `G_SYNC_CMD_ROT.py`

- **Foto / Identidade:** `G_SYNC_CMD_ROT.py`
- **Caminho no Disco:** `gates/G_SYNC_CMD_ROT.py`

- **Missão de Segurança:** Bloqueia a forma incompleta de sincronização de componentes em docs vivos
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Bloqueia a forma incompleta de sincronização de componentes em docs vivos
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
 - ▶ **D8. O que o Estágio Processa:** técnica: expressão regular.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 7 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_sync_cmd_rot.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

59. Guarda Incorrúptível: `G_TEMPLATE_FORGE_ROT.py`

- **Foto / Identidade:** `G_TEMPLATE_FORGE_ROT.py`
- **Caminho no Disco:** `gates/G_TEMPLATE_FORGE_ROT.py`
- **Missão de Segurança:** Portão determinístico de integridade dos templates do aidd-forge.
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Portão determinístico de integridade dos templates do aidd-forge.
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
 - ▶ **D8. O que o Estágio Processa:** técnica: leitura direta de arquivos.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** nenhum bloco `try`: erro inesperado derruba o portão (e o commit reprova).
 - ▶ **D12. Observabilidade e Frugalidade:** 8 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_template_forge_rot.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

60. Guarda Incorruptível: `G_TEMPLATE_TANSTACK_OFFLINE.py`

- **Foto / Identidade:** `G_TEMPLATE_TANSTACK_OFFLINE.py`
- **Caminho no Disco:** `gates/G_TEMPLATE_TANSTACK_OFFLINE.py`
- **Missão de Segurança:** `G_TEMPLATE_TANSTACK_OFFLINE.py`
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** `G_TEMPLATE_TANSTACK_OFFLINE.py`
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
 - ▶ **D8. O que o Estágio Processa:** técnica: leitura direta de arquivos.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 5 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_template_tanstack_offline.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

61. Guarda Incorruptível: `G_TESTES_REAIS.py`

- **Foto / Identidade:** `G_TESTES_REAIS.py`
- **Caminho no Disco:** `gates/G_TESTES_REAIS.py`
- **Missão de Segurança:** Roda pytest de verdade em cada tools/ e falha (exit 1) se
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** grava ou remove arquivos (`open(w)`, `os.remove`); executa subprocessos, cujos efeitos não entram nesta contagem.
 - ▶ **D4. Componentes e Fractalidade:** importa módulos do repositório: `_escopo_commit`
 - ▶ **D5. Visão e Escopo:** Roda pytest de verdade em cada tools/ e falha (exit 1) se
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
 - ▶ **D8. O que o Estágio Processa:** técnica: subprocesso.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** 8 bloco(s) `try` no código.
 - ▶ **D12. Observabilidade e Frugalidade:** 39 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_testes_reais.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

62. Guarda IncorrupΤίvel: `G_TRANSACTION_LOG_LRU.py`

- **Foto / Identidade:** `G_TRANSACTION_LOG_LRU.py`
- **Caminho no Disco:** `gates/G_TRANSACTION_LOG_LRU.py`
- **Missão de Segurança:** Verificação determinística (AST + hashes SHA-256) da entrega do item
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - **D5. Visão e Escopo:** Verificação determinística (AST + hashes SHA-256) da entrega do item
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** o repositório inteiro.
 - **D8. O que o Estágio Processa:** técnica: AST Python, hash criptográfico.
 - **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - **D11. Tratamento de Exceções e Fallback:** 3 bloco(s) `try` no código.
 - **D12. Observabilidade e Frugalidade:** 23 saída(s) `print` para o terminal.
 - **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_transaction_log_lru.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

63. Guarda IncorrupΤίvel: `G_UNIVERSAL_HARNESS.py`

- **Foto / Identidade:** `G_UNIVERSAL_HARNESS.py`
- **Caminho no Disco:** `gates/G_UNIVERSAL_HARNESS.py`
- **Missão de Segurança:** Audita a garantia SINE QUA NON de que todo componente, skill, MCP e hook
- **Lente 15-D deste Quality Gate:**
 - **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - **D4. Componentes e Fractalidade:** importa módulos do repositório: `gestor_componentes`, `gestor_dependencias`
 - **D5. Visão e Escopo:** Audita a garantia SINE QUA NON de que todo componente, skill, MCP e hook
 - **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - **D7. O que o Estágio Recebe:** o repositório inteiro.
 - **D8. O que o Estágio Processa:** técnica: leitura direta de arquivos.
 - **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
 - **D12. Observabilidade e Frugalidade:** 15 saída(s) `print` para o terminal.

- ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_universal_harness.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

64. Guarda Incorrupível: `G_USER_FACING_PTBR.py`

- **Foto / Identidade:** `G_USER_FACING_PTBR.py`
- **Caminho no Disco:** `gates/G_USER_FACING_PTBR.py`
- **Missão de Segurança:** Bloqueia jargão não explicado em superfícies lidas pelo usuário leigo
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** sem flags (roda sobre o repositório inteiro). Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** nenhuma chamada de escrita ou remoção de arquivo no próprio código.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Bloqueia jargão não explicado em superfícies lidas pelo usuário leigo
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** o repositório inteiro.
 - ▶ **D8. O que o Estágio Processa:** técnica: expressão regular.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).
 - ▶ **D10. Orquestração e Topologia:** acionado pelo pre-commit.
 - ▶ **D11. Tratamento de Exceções e Fallback:** nenhum bloco `try`: erro inesperado derruba o portão (e o commit reprova).
 - ▶ **D12. Observabilidade e Frugalidade:** 7 saída(s) `print` para o terminal.
 - ▶ **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_user_facing_ptbr.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
 - ▶ **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
 - ▶ **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

65. Guarda Incorrupível: `G_ZERO_HEADLESS.py`

- **Foto / Identidade:** `G_ZERO_HEADLESS.py`
- **Caminho no Disco:** `gates/G_ZERO_HEADLESS.py`
- **Missão de Segurança:** Gate de Enforcement Real contra Subagentes Headless e Concorrência Paralela
- **Lente 15-D deste Quality Gate:**
 - ▶ **D1. Contratos e Regras:** Lei #2 (saída binária) e Lei #13 (prova que morde).
 - ▶ **D2. Input e Gatilhos:** flags `--dangerously-force-headless`. Roda automaticamente no commit via `.pre-commit-config.yaml`.
 - ▶ **D3. Raio de Impacto e Isolamento:** grava ou remove arquivos (`unlink`, `write_text`); executa subprocessos, cujos efeitos não entram nesta contagem.
 - ▶ **D4. Componentes e Fractalidade:** não importa módulos do repositório (autocontido).
 - ▶ **D5. Visão e Escopo:** Gate de Enforcement Real contra Subagentes Headless e Concorrência Paralela
 - ▶ **D6. O que o Estágio Faz:** auditar o que D5 descreve numa única passada (portão de estágio único).
 - ▶ **D7. O que o Estágio Recebe:** os arquivos passados por argumento ou, sem eles, o repositório.
 - ▶ **D8. O que o Estágio Processa:** técnica: subprocesso.
 - ▶ **D9. O que o Estágio Entrega:** `exit 0` (aprovado) ou `exit 1` (bloqueado).

- **D10. Orquestração e Topologia:** acionado pelo pre-commit.
- **D11. Tratamento de Exceções e Fallback:** 1 bloco(s) `try` no código.
- **D12. Observabilidade e Frugalidade:** 23 saída(s) `print` para o terminal.
- **D13. Quality Gates (Portões):** teste próprio: `gates/test_g_zero_headless.py`; o meta-portão `G_PORTAO_PROVA_QUE_MORDE` confere que ele reprova de verdade.
- **D14. Critério de Rejeição (Rollback):** `exit 1` bloqueia o commit.
- **D15. Output Consolidado e Handoff:** o código de saída e as mensagens no terminal.

PARTE IV: OS 132 COMPONENTES AGNÓSTICOS (PEÇAS UNIVERSAIS LEGO)

Na Festa

Imagine peças de LEGO de altíssima precisão. Não importa se você está montando na mesa azul (Claude), na mesa verde (Gemini), na mesa cinza (Cursor) ou na mesa preta (OpenCode). As peças encaixam com o mesmo clique suave e a mesma firmeza matemática. Nenhuma mesa fica com uma peça diferente da outra.

Na Casa (A Sincronização dos 132 Componentes)

- **Fonte Canônica Única:** [componentes/compartilhado/](#)
- **Ambientes Espelhados (Harnesses):** [.agents/](#), [.claude/](#), [.codebuddy/](#), [.codex/](#), [.cursor/](#), [.gemini/](#), [.mimocode/](#), [.opencode/](#).
- **Garantia de Integridade:** Validação criptográfica SHA-256 via `python ecossistema.py components verify --tipo todos`. Se um único byte divergir entre qualquer harness e a fonte canônica, o sistema aponta drift e bloqueia o commit.

As 6 Famílias dos 132 Componentes:

1. **Slash Commands Canônicos** ([componentes/compartilhado/comandos/](#)): 19 especificações de comando em Markdown ([/pure](#), [/forge](#), [/melhoria](#), etc.).
2. **Campainhas e Travas Git** ([componentes/compartilhado/hooks/](#)): 16 itens — travas de pré-commit e verificação de regras.
3. **Escudo de Segurança** ([componentes/compartilhado/security/](#)): 2 itens — detectores de vazamento de segredos e chaves.
4. **Habilidades Atômicas** ([componentes/compartilhado/skills/](#)): 44 pastas de skills padronizadas.
5. **Esquemas e Contratos Formais** ([componentes/compartilhado/specs/](#)): 11 esquemas JSON Draft-7 para handoff formal entre módulos.
6. **Núcleo Compartilhado de Código** ([componentes/compartilhado/src-core/](#)): 40 itens — utilitários de missão crítica ([escritor_atomico.py](#), [result.py](#), [assinatura_manifesto.py](#)).

PARTE V: AS AUDITORIAS EM ANDAMENTO (OS CICLOS 4F)

Na Festa

A fábrica não só fabrica brinquedos: de tempos em tempos ela para uma oficina inteira para revisão. Um inspetor anota os defeitos, o chefe escreve a ordem de serviço, um mecânico conserta e o inspetor volta para conferir. Enquanto o dono não assina a liberação, a oficina consertada fica numa garagem ao lado, sem voltar para a linha de produção.

Na Casa (o estado real em 4 de outubro de 2026)

Cada rodada de auditoria de uma ferramenta é um ciclo em `docs/auditoria/<ferramenta>/ciclo-NN/`. A tabela abaixo foi montada pelo gerador olhando os arquivos de cada ciclo e o git, não copiada de relatório: a Fase 3 conta como feita quando existe `RELATORIO-CONSTRUTOR.md` ou commits na branch de evolução, e "aprovado" quer dizer que existe `refs/aidd/aprovavel/...` (o `gate_final` passou). Ciclos no formato antigo são anteriores a esse registro e aparecem como "não" mesmo já mesclados.

Ciclo	Fases	Onde está o trabalho	<code>gate_final</code> aprovado
<code>agilidade-gates/ciclo-01</code>	1 Inspe- tor pen- dente · 2 Arqui- teto feito · 3 Cons- trutor pen- dente · 4 Re- torno pen- dente	ainda não construído	não
<code>aidd-diagnose/ciclo-01</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Cons- trutor feito · 4 Retorno feito	na <code>main</code>	não
<code>aidd-enterprise/ciclo-01</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Cons- trutor feito · 4	na <code>main</code>	não

Ciclo	Fases	Onde está o trabalho	<code>gate_final</code> aprovado
	Retorno feito		
<code>aidd- forge/ ciclo- 01</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Cons- trutor feito · 4 Retorno feito	na <code>main</code>	não
<code>aidd- forge/ ciclo- 02</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Cons- trutor feito · 4 Retorno feito	na <code>main</code>	não
<code>aidd- freedom/ ciclo- 02</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Cons- trutor feito · 4 Retorno feito	na <code>main</code>	não
<code>aidd- grill/ ciclo- 01</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Cons- trutor feito · 4 Retorno feito	na <code>main</code>	não
<code>aidd- melhoria/ ciclo- 01</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Cons- trutor feito · 4 Retorno feito	na <code>main</code>	não
<code>aidd- melhoria/ ciclo- 02</code>	1 Inspe- tor feito · 2 Ar- quiteto	na <code>main</code>	não

Ciclo	Fases	Onde está o trabalho	<code>gate_final</code> aprovado
	feito · 3 Construtor feito · 4 Retorno feito		
<code>aidd-skills/ciclo-01</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Con- strutor feito · 4 Retorno feito	na <code>main</code>	não
<code>aidd-skills/ciclo-02</code>	1 Inspe- tor pen- dente · 2 Arqui- teto feito · 3 Con- strutor pen- dente · 4 Re- torno pen- dente	ainda não construído	não
<code>aidd-spec/ciclo-01</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Con- strutor feito · 4 Retorno feito	na <code>main</code>	não
<code>aidd-tdd/ciclo-01</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Con- strutor feito · 4 Retorno feito	na <code>main</code>	não
<code>aidd-tickets/ciclo-01</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Con- strutor	na <code>main</code>	não

Ciclo	Fases	Onde está o trabalho	<code>gate_final</code> aprovado
	feito · 4 Retorno feito		
<code>cbm-lazy-proxy/ciclo-01</code>	1 Inspe- tor pen- dente · 2 Arqui- teto feito · 3 Cons- trutor pen- dente · 4 Re- torno pen- dente	ainda não construído	não
<code>fronteiras-ferramentas/ciclo-01</code>	1 Inspe- tor pen- dente · 2 Arqui- teto feito · 3 Cons- trutor pen- dente · 4 Re- torno pen- dente	ainda não construído	não
<code>mapa-pecas/ciclo-01</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Cons- trutor feito · 4 Retorno feito	na <code>main</code>	não
<code>mapa-pecas/ciclo-02</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Cons- trutor feito · 4 Retorno feito	na <code>main</code>	não
<code>mapa-pecas/ciclo-03</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Cons-	na <code>main</code>	não

Ciclo	Fases	Onde está o trabalho	<code>gate_final</code> aprovado
	trutor feito · 4 Retorno feito		
<code>mapa- pecas/ ciclo- 04</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Cons- trutor feito · 4 Retorno feito	na <code>main</code>	não
<code>meus- prompts/ ciclo- 01</code>	1 Inspe- tor pen- dente · 2 Arqui- teto feito · 3 Cons- trutor pen- dente · 4 Re- torno pen- dente	ainda não construído	não
<code>skills- ddd/ ciclo- 01</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Cons- trutor feito · 4 Retorno feito	na <code>main</code>	não
<code>skills- pocock/ ciclo- 01</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Cons- trutor feito · 4 Retorno feito	na <code>main</code>	não
<code>template- tanstack- offline/ ciclo- 01</code>	1 Inspe- tor feito · 2 Ar- quiteto feito · 3 Cons- trutor feito · 4	na <code>main</code>	não

Ciclo	Fases	Onde está o trabalho	<code>gate_final</code> aprovado
	Retorno feito		

As 15 dimensões da Lente 15-D, na ordem em que o Inspetor as preenche:

1. **D1. Contratos e Regras**
2. **D2. Input e Gatilhos**
3. **D3. Raio de Impacto e Isolamento**
4. **D4. Componentes e Fractalidade**
5. **D5. Visão e Escopo**
6. **D6. O que o Estágio Faz**
7. **D7. O que o Estágio Recebe**
8. **D8. O que o Estágio Processa**
9. **D9. O que o Estágio Entrega**
10. **D10. Orquestração e Topologia**
11. **D11. Tratamento de Exceções e Fallback**
12. **D12. Observabilidade e Frugalidade**
13. **D13. Quality Gates (Portões)**
14. **D14. Critério de Rejeição (Rollback)**
15. **D15. Output Consolidado e Handoff**

Nada disso entra na `main` sem aprovação humana: `python scripts/orquestrador_4f.py --manifest <json> --aprovar.`

Errata das Fichas de 2026-09-22

As fichas da Parte I são o registro histórico da auditoria de 2026-09-22. Estes nomes estavam errados nelas e foram corrigidos ao gerar este livro (o arquivo histórico não foi alterado):

- **aidd-forge:** `G_BLOQUEAR_SEGREDO` → `G_BLOQUEAR_SEGREDOS`
- **aidd-forge:** `G_CYBERSECURITY` → `G_CYBERSECURITY_OWASP`
- **aidd-planner:** `aidd-generator`, `aidd-factory`, `aidd-bridge` → `aidd-pure`, `aidd-open`, `aidd-freedom`
- **aidd-open:** `G_FACTORY_INPUT` (validação estrita do schema do plano de infraestrutura), `G_FACTORY_OUTPUT` (verificação do manifesto de entrega), `G_FACTORY_DETERMINISTIC` (garantia de zero LLM nas fases 1, 4, 5 e 6). → `G_FACTORY_ANALYSIS`, `G_FACTORY_COMPOSE`, `G_FACTORY_ENV`, `G_FACTORY_INIT_DB`, `G_FACTORY_INTEGRATION` e `G_FACTORY_MVP` (em `tools/aidd-open/gates/`). `G_FACTORY_INPUT`, `G_FACTORY_OUTPUT` e `G_FACTORY_DETERMINISTIC` são rótulos de invariante no `AGENTS.md`, não arquivos de portão.
- **aidd-ops:** catálogo de ferramentas (`data/catalogo_ferramentas.json`) → requisitos de recursos (`data/requisitos_recursos.json`)

Epílogo: A Orquestra Perfeita

Quando você junta as **8 Ferramentas Macro**, as **44 Micro-Ferramentas**, os **65 Quality Gates** e os **132 Componentes Agnósticos**, o que você tem não é apenas um projeto de software: você tem uma **catedral digital determinística**.

No Ecossistema AIDD, nenhuma linha de código nasce sem plano, nenhuma ferramenta opera sem contrato e nenhum portão se abre sem que a prova matemática tenha sido atendida.

Fim da Auditoria Canônica — 4 de outubro de 2026.